



离散数学 II

一道路与回路的经典问题1

周旻
清华大学软件学院
软件工程与系统研究所

2026年3月26日

第二章 道路与回路

- 道路与回路的定义和相关概念
- 道路与回路的判定方法
- 欧拉道路与回路
- 哈密顿道路与回路
- 旅行商问题
- 最短路径
- 关键路径
- 中国邮路

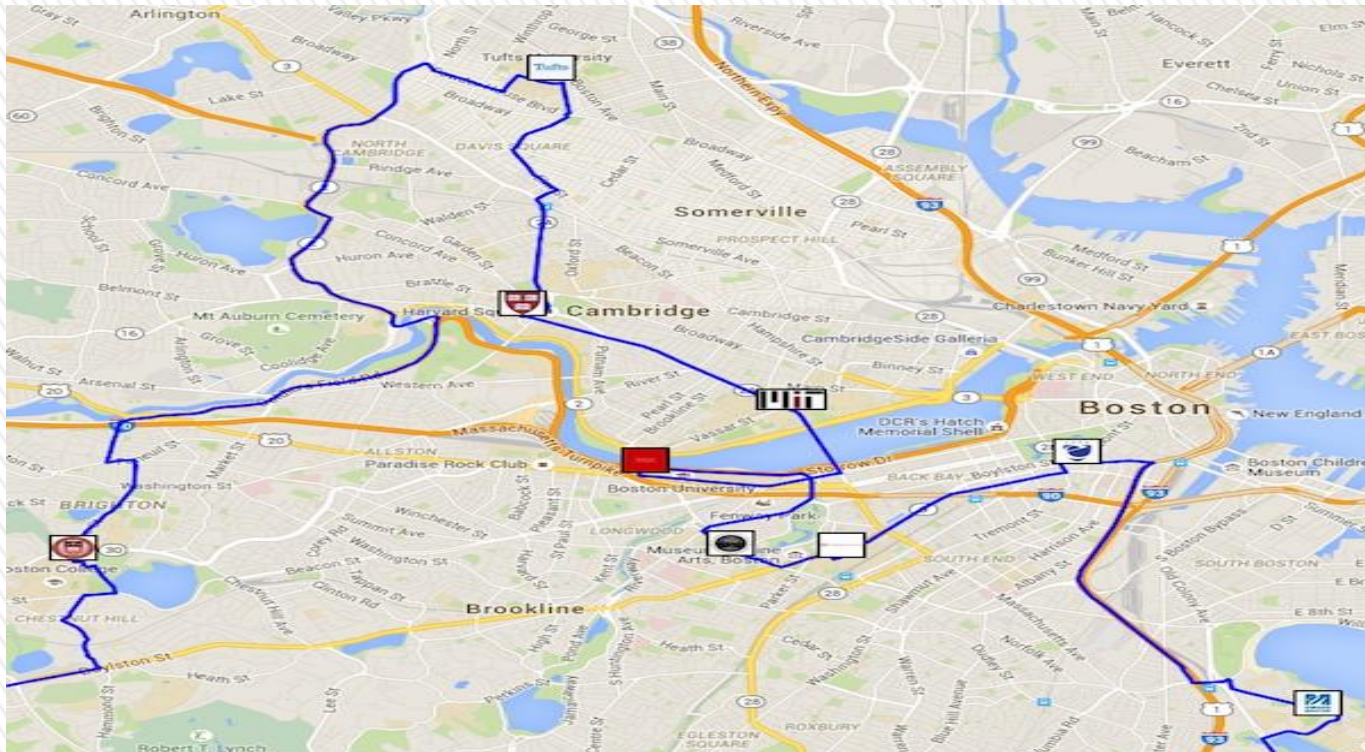
旅行商问题

旅行商问题

- H-圈不涉及边的长度，但是在许多实际问题中，每条边都可以有它的权。边权可以是该路的长度、旅行的费用或所需的时间。
- 旅行商问题(TSP--traveling salesman problem):
 - 货郎担问题：一个商人欲到 n 个城市推销商品，每两个城市 i 和 j 之间的距离为 d_{ij} ，如何选择一条道路使得商人每个城市走一遍后回到起点，且所走路径最短。
- 图论语言：给定一个正权完全图, 求具有总长最短的H-圈。

旅行商问题

■ 旅行商问题（又叫货郎担问题）



shortest road trip to visit all 647 campuses on Forbes' list of America's Top Colleges.

旅行商问题

■ 旅行商问题（又叫货郎担问题）

典型的NP-完全问题

专门网站

<http://www.math.uwaterloo.ca/tsp/>

可以找到各种规模、很多国家的测试数据，以及目前已有的TSP问题研究进展，包括源码

United States TSP	\$500 Prize for best tour through 115,475 US cities.
Scientific American	Short piece on Yogi Berra and the TSP
Travelling Salesman	Thriller movie centered around a solution of the TSP
Stephen Colbert	49,512-city tour of Colbert Nation via the iPhone App.
Mona Lisa TSP	\$1,000 Prize for a 100,000-city challenge problem.
Google Maps	Plot an optimal TSP tour with a Google interface.
pla85900	Solution of a 85,900-city TSP.
Iowa Tour	Optimal route for a 99-county campaign tour.

旅行商问题

Year	Research Team	Size of Instance	Name
1954	G. Dantzig, R. Fulkerson, and S. Johnson	49 cities	<u>dantzig42</u>
1971	M. Held and R.M. Karp	64 cities	64 random points
1975	P.M. Camerini, L. Fratta, and F. Maffioli	67 cities	67 random points
1977	M. Grötschel	120 cities	<u>gr120</u>
1980	H. Crowder and M.W. Padberg	318 cities	<u>lin318</u>
1987	M. Padberg and G. Rinaldi	532 cities	<u>att532</u>
1987	M. Grötschel and O. Holland	666 cities	<u>gr666</u>
1987	M. Padberg and G. Rinaldi	2,392 cities	<u>pr2392</u>
1994	D. Applegate, R. Bixby, V. Chvátal, and W. Cook	7,397 cities	<u>pla7397</u>
1998	D. Applegate, R. Bixby, V. Chvátal, and W. Cook	13,509 cities	<u>usa13509</u>
2001	D. Applegate, R. Bixby, V. Chvátal, and W. Cook	15,112 cities	<u>d15112</u>
2004	D. Applegate, R. Bixby, V. Chvátal, W. Cook, and K. Helsgaun	24,978 cities	<u>sw24798</u>



> [Home](#)

[The Problem](#)

[History](#)

[Applications](#)

[Solving a TSP](#)

[World Records](#)

[Gallery](#)

[TSP Games](#)

[Google Maps](#)

[Concorde](#)

[Test Data](#)

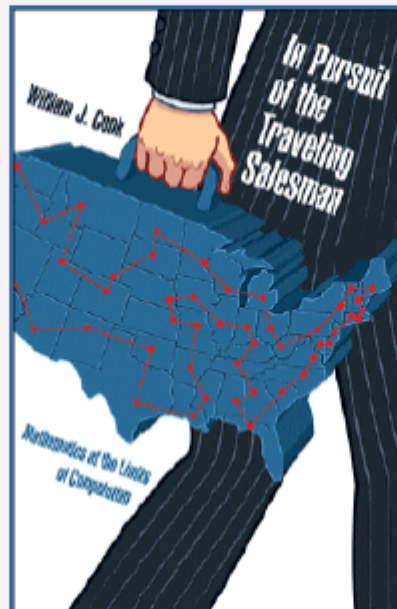
[News](#)

[TSP Book](#)

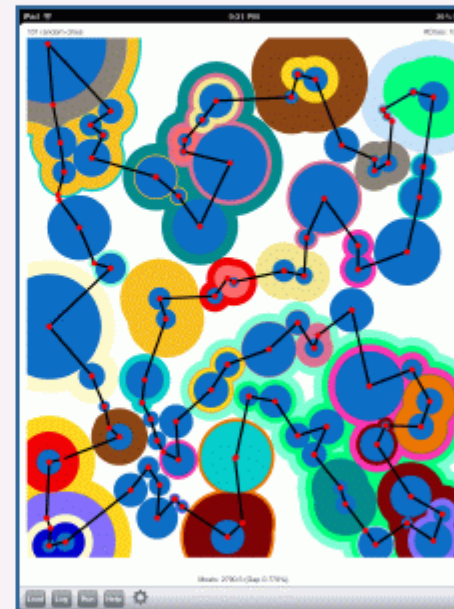
[Search Site](#)

The Traveling Salesman Problem

The Traveling Salesman Problem is one of the most intensively studied problems in computational mathematics. These pages are devoted to the history, applications, and current research of this challenge of finding the shortest route visiting each member of a collection of locations and returning to your starting point.



[TSP Book](#)
[\\$18.45 at Amazon.com](#)
[Chapter 1 as pdf file](#)
[Facebook Page](#)



[TSP iPhone/iPad App](#)
[iTunes Preview](#)
[App Support Page](#)
[NY Times Article](#)

旅行商问题

■ 旅行商问题（又叫货郎担问题）

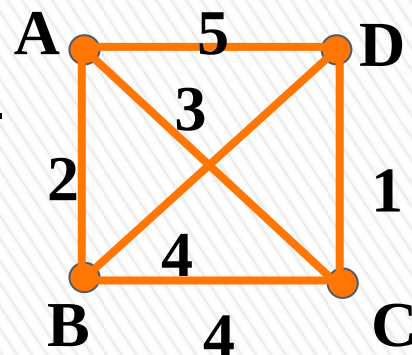
给定一个正权完全图, 求具有总长最短的H-圈
求解方法

- 枚举法

n个节点的完全图有多少个不同的H回路？

$$\frac{1}{2}(n-1)! \quad \text{由Stirling公式, } n! \approx \sqrt{2n\pi} \left(\frac{n}{e}\right)^n$$

2001年解决了德国15112个城市之间的TSP问题, 共使用了美国Rice大学和普林斯顿大学之间网络互连的, 由速度为500MHz 的Compaq EV6 Alpha处理器组成的110台计算机, 所有计算机花费的时间之和为22.6年。

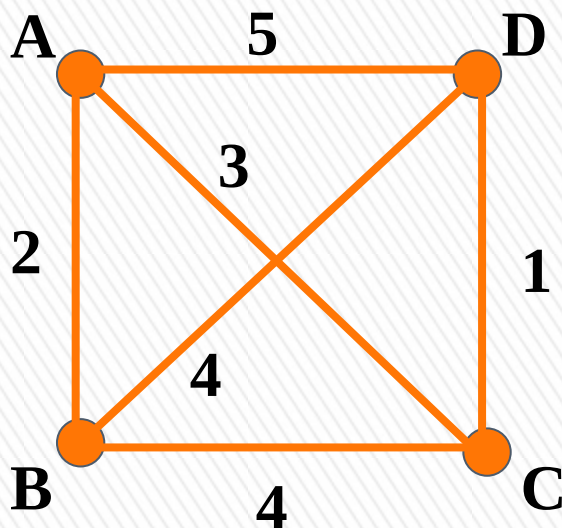


旅行商问题

■ 旅行商问题（又叫货郎担问题）

给定一个正权完全图，求具有总长最短的H-圈。

例：

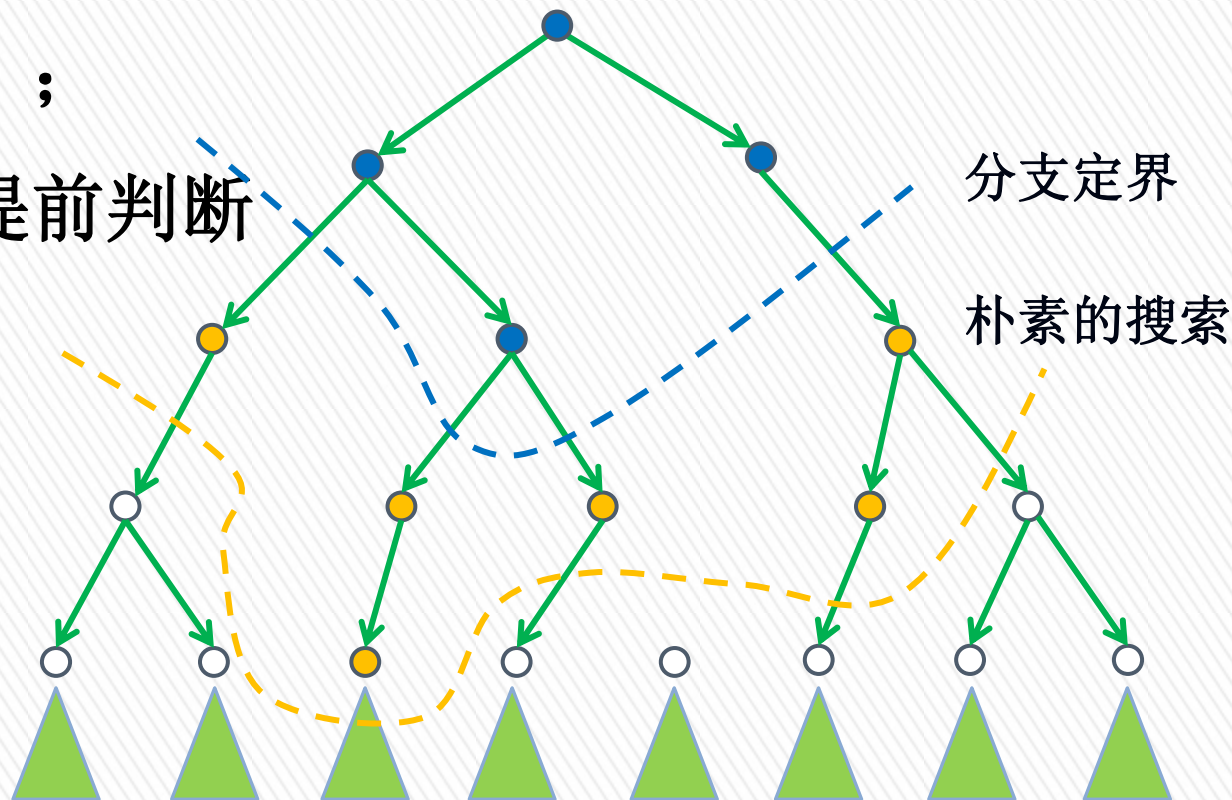


最短的H回路是(A, B, D, C, A)，长为10

分支定界

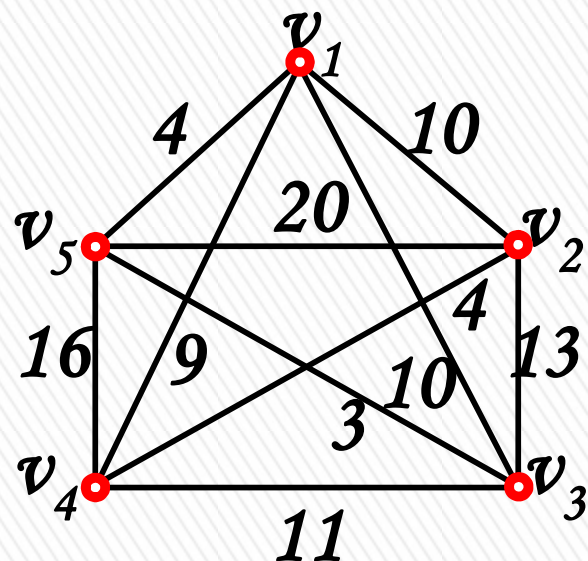
- 减少搜索的范围：搜索过程中，提前判断“继续当前的搜索分支是否可能产生最优解”，如果不可能则提前结束**当前分支**的搜索（继续搜索其他分支）；

- 关键：如何提前判断



旅行商问题的分支定界法

实例：五个城市间交通费用图，求最少费用旅游一遍并回原点的路线。

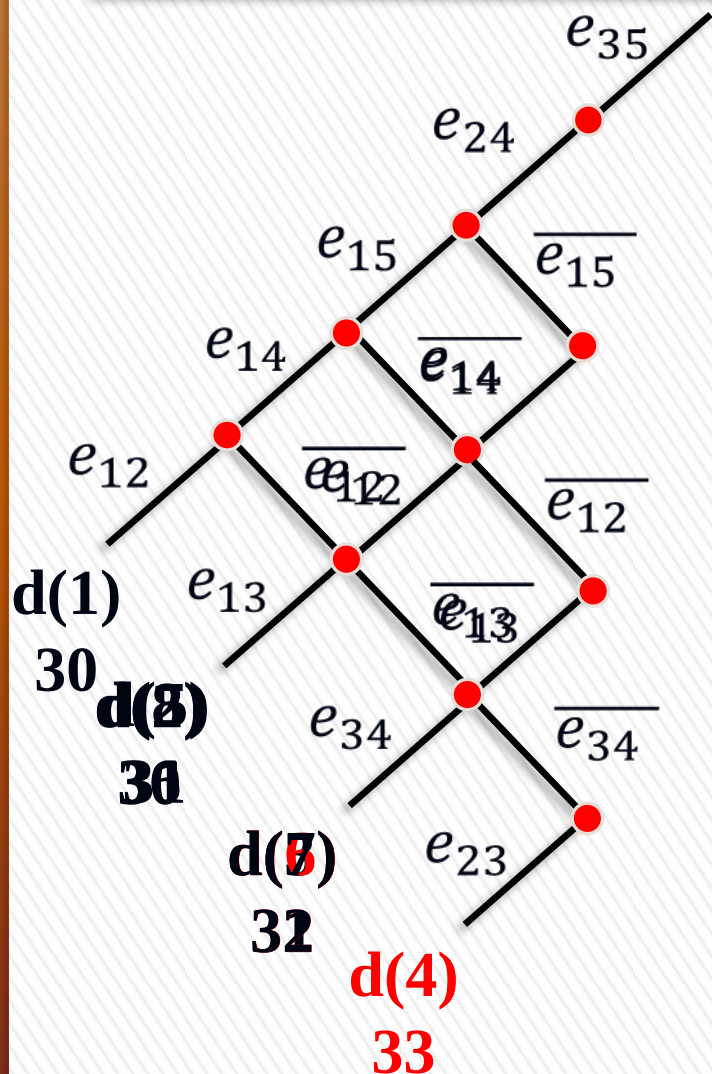


(1) 首先将边权由小到大排序。初始界 $d_0 \leftarrow \infty$.

$v_i v_j$:	$v_5 v_3$	$v_4 v_2$	$v_5 v_1$	$v_1 v_4$	$v_1 v_2$	$v_1 v_3$	$v_3 v_4$	$v_2 v_3$	$v_5 v_4$	$v_5 v_2$
w_{ij} :	3	4	4	9	10	10	11	13	16	20

(2) 采用DFS方法和分支判断步骤如下：

旅行商问题的分支定界法



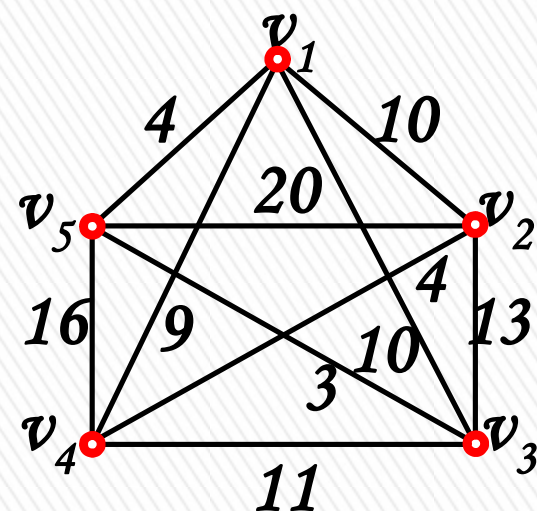
$$l_y \begin{pmatrix} 3 & 4 & 4 & 9 & 10 & 10 & 11 & 13 & 16 & 20 \\ e_y \begin{pmatrix} e_{35} & e_{24} & e_{15} & e_{14} & e_{12} & e_{13} & e_{34} & e_{23} & e_{45} & e_{25} \end{pmatrix} \end{pmatrix}$$

$d(1)=30$ $d(2)=30$

$d(3)=31$ $d(4)=33$

$d(5)=31$ $d(6)=32$

$d(7)=32$ $d(8)=36$



最优解是 $d(6)=32$

即 $(e_{35}, e_{51}, e_{12}, e_{24}, e_{43})$

旅行商问题的分支定界法

实例: 五个城市间交通费用图，求最少费用旅游一遍并回原点的路线。

(1) 首先将边权由小到大排序。初始界 $d_0 \leftarrow \infty$ 。

$v_i v_j$:	$v_5 v_3$	$v_4 v_2$	$v_5 v_1$	$v_1 v_4$	$v_1 v_2$	$v_1 v_3$	$v_3 v_4$	$v_2 v_3$	$v_5 v_4$	$v_5 v_2$
w_{ij} :	3	4	4	9	10	10	11	13	16	20

(2) 采用DFS方法和分支判断步骤如下:

$d(s_1) = d(v_5 v_3, v_4 v_2, v_5 v_1, v_1 v_4, v_1 v_2) = 30$. (不是H-cycle).

$d(s_2) = d(v_5 v_3, v_4 v_2, v_5 v_1, v_1 v_4, v_1 v_3) = 30$. (不是H-cycle).

$d(s_3) = d(v_5 v_3, v_4 v_2, v_5 v_1, v_1 v_4, v_3 v_4) = 32$. (不是H-cycle).

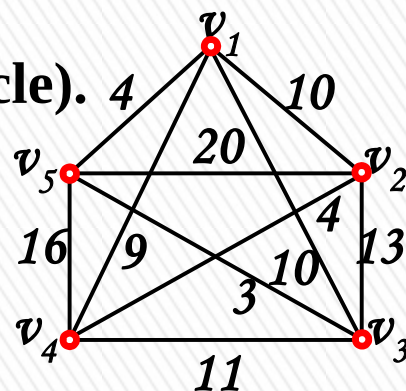
$d(s_4) = d(v_5 v_3, v_4 v_2, v_5 v_1, v_1 v_4, v_2 v_3) = 33$. $d_0 \leftarrow 33$.

$d(s_5) = d(v_5 v_3, v_4 v_2, v_5 v_1, v_1 v_2, v_1 v_3) = 31$. (不是H-cycle).

$d(s_6) = d(v_5 v_3, v_4 v_2, v_5 v_1, v_1 v_2, v_3 v_4) = 32$. $d_0 \leftarrow 32$.

$d(s_7) = d(v_5 v_3, v_4 v_2, v_4 v_1, v_1 v_2, v_1 v_3) = 36 > d_0$.

$\therefore$ 最优解为 $d(s_6) = 32$.



分支定界法

基本思想

对**有约束条件的最优化问题**的所有可行解（数目有限）空间进行搜索。

- **分支**：把全部可行的解空间不断分割为越来越小的子集；
- **定界**：为每个子集内的解的值计算一个下界或上界；
- **剪枝**：在每次分支后，对凡是界限超出已知可行解值那些子集不再做进一步分支。

一直进行到找出可行解为止，该可行解的值不大于任何子集的界限。因此这种算法一般可以求得最优解。

已成功应用于整数规划问题、生产进度表、旅行商问题、选址、背包问题以及可行解的数目为有限的许多其它问题。

旅行商问题的近似算法

例：已知图 G 的权矩阵，求解其旅行商问题的近似解

0	18	35	25	27
18	0	23	21	19
35	23	0	17	28
25	21	17	0	24
27	19	28	24	0

旅行商问题的解需要满足的条件

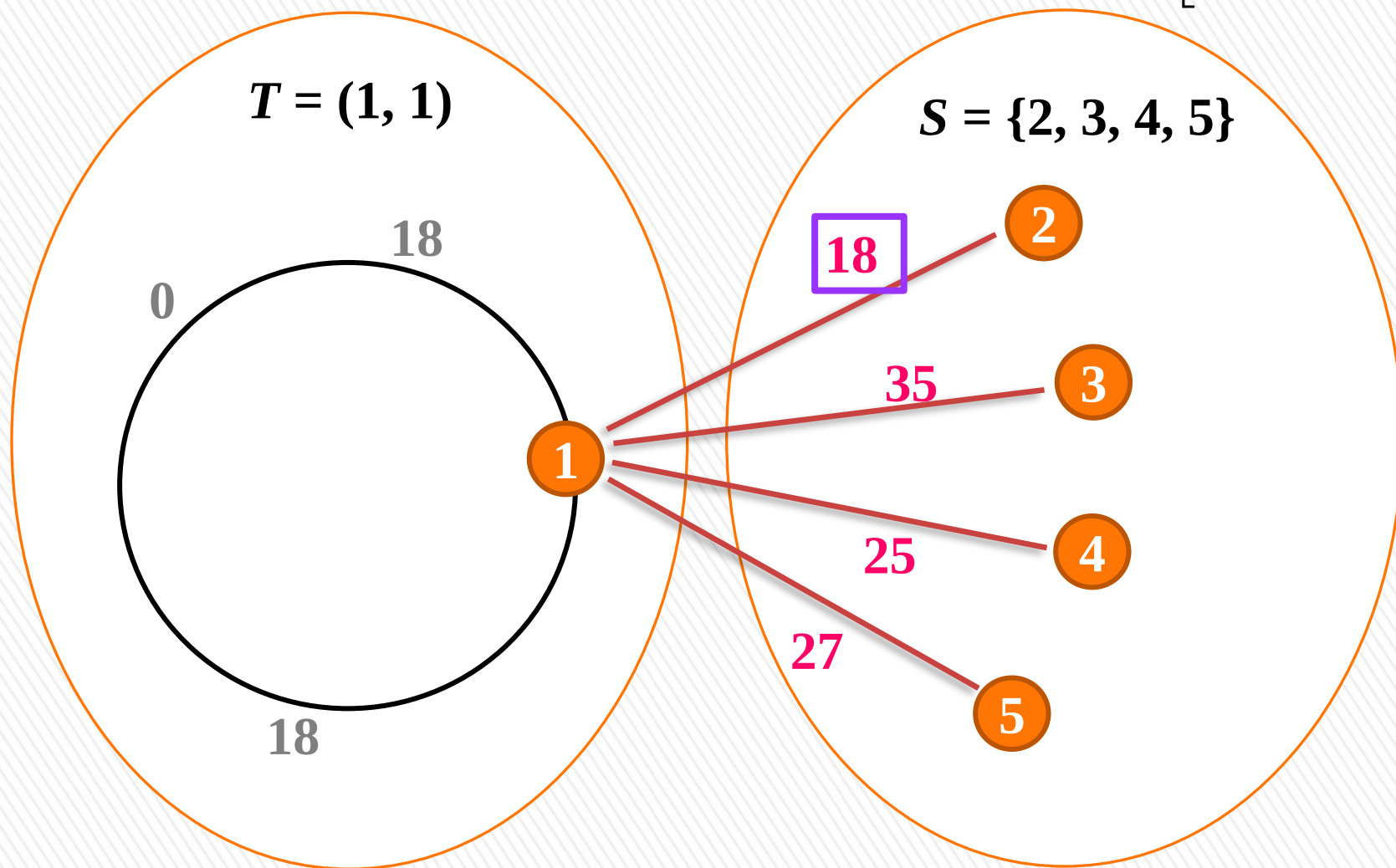
(1) 回路

(2) 每个节点只能出现一次

(3) 边权总和最小

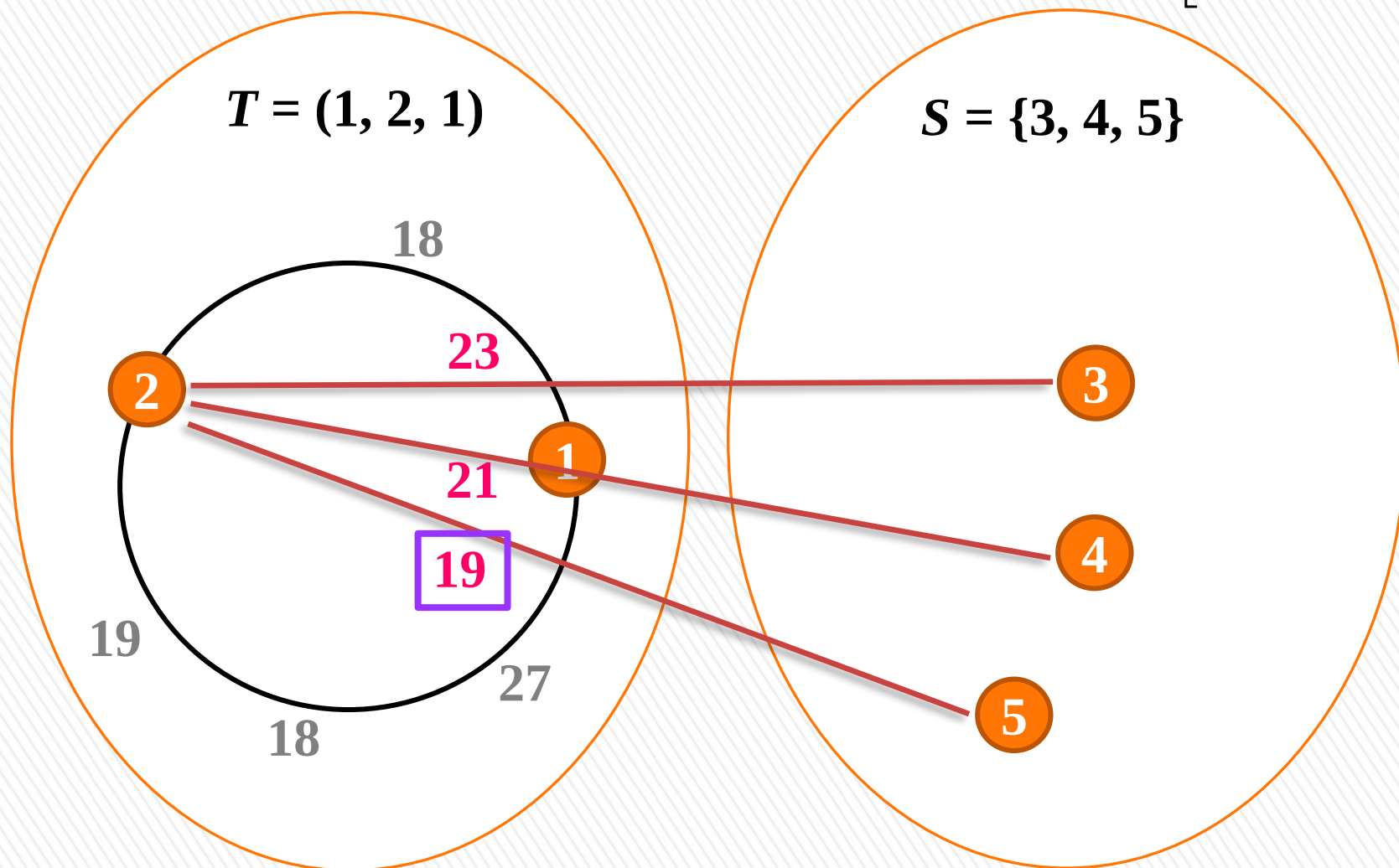
旅行商问题的近似算法

0	18	35	25	27
18	0	23	21	19
35	23	0	17	28
25	21	17	0	24
27	19	28	24	0



旅行商问题的近似算法

0	18	35	25	27
18	0	23	21	19
35	23	0	17	28
25	21	17	0	24
27	19	28	24	0

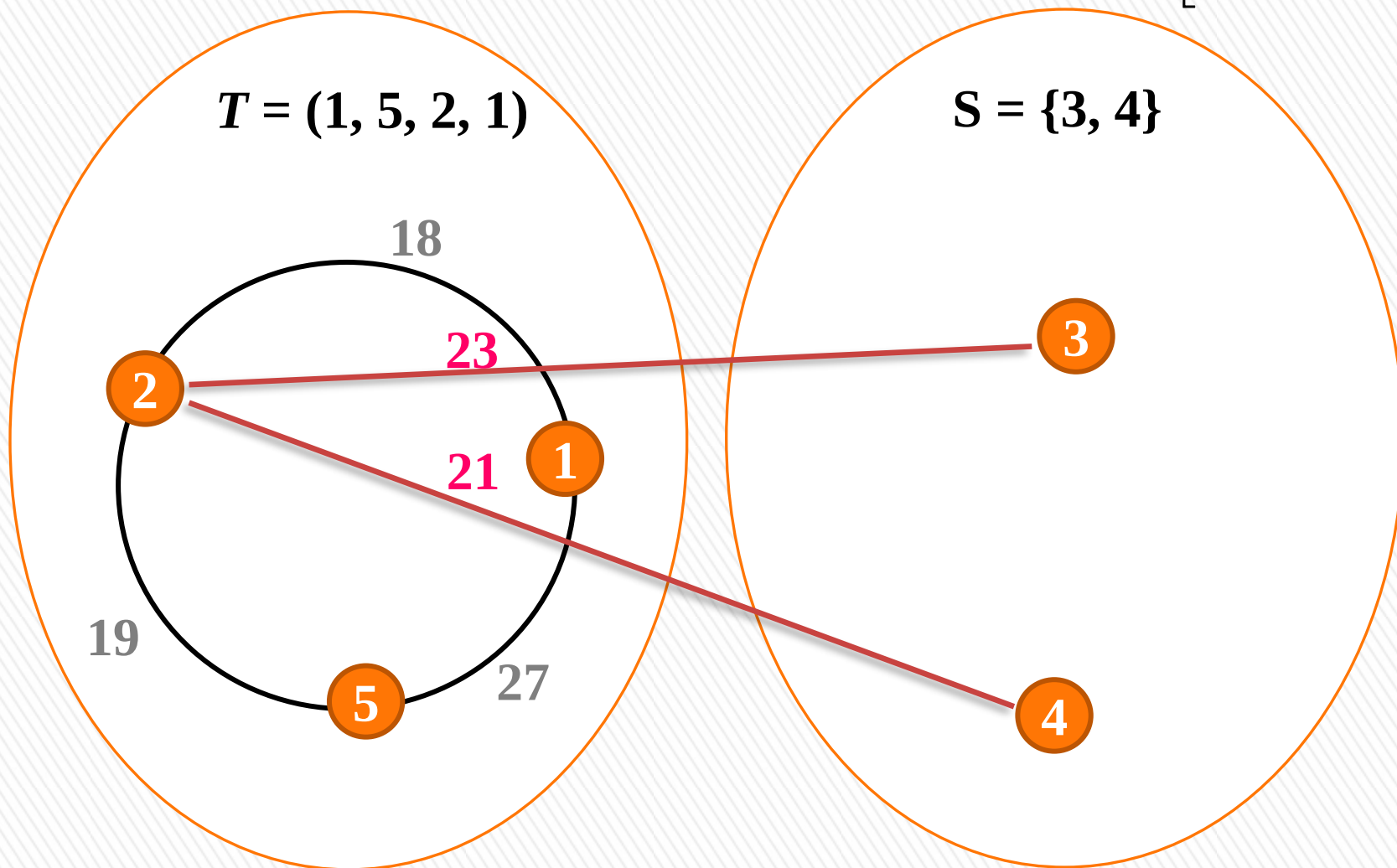


旅行商问题的近似算法

0	18	35	25	27
18	0	23	21	19
35	23	0	17	28
25	21	17	0	24
27	19	28	24	0

$T = (1, 5, 2, 1)$

$S = \{3, 4\}$



旅行商问题

■ 贪心算法

将问题的求解过程看作是一系列选择, 每次选择一个输入, 每次选择都是当前状态下的最好选择(局部最优解). 每作一次选择后, 所求问题会简化为一个规模更小的子问题. 从而通过每一步的最优解(希望)逐步达到整体的最优解。

达到最优解的前提:

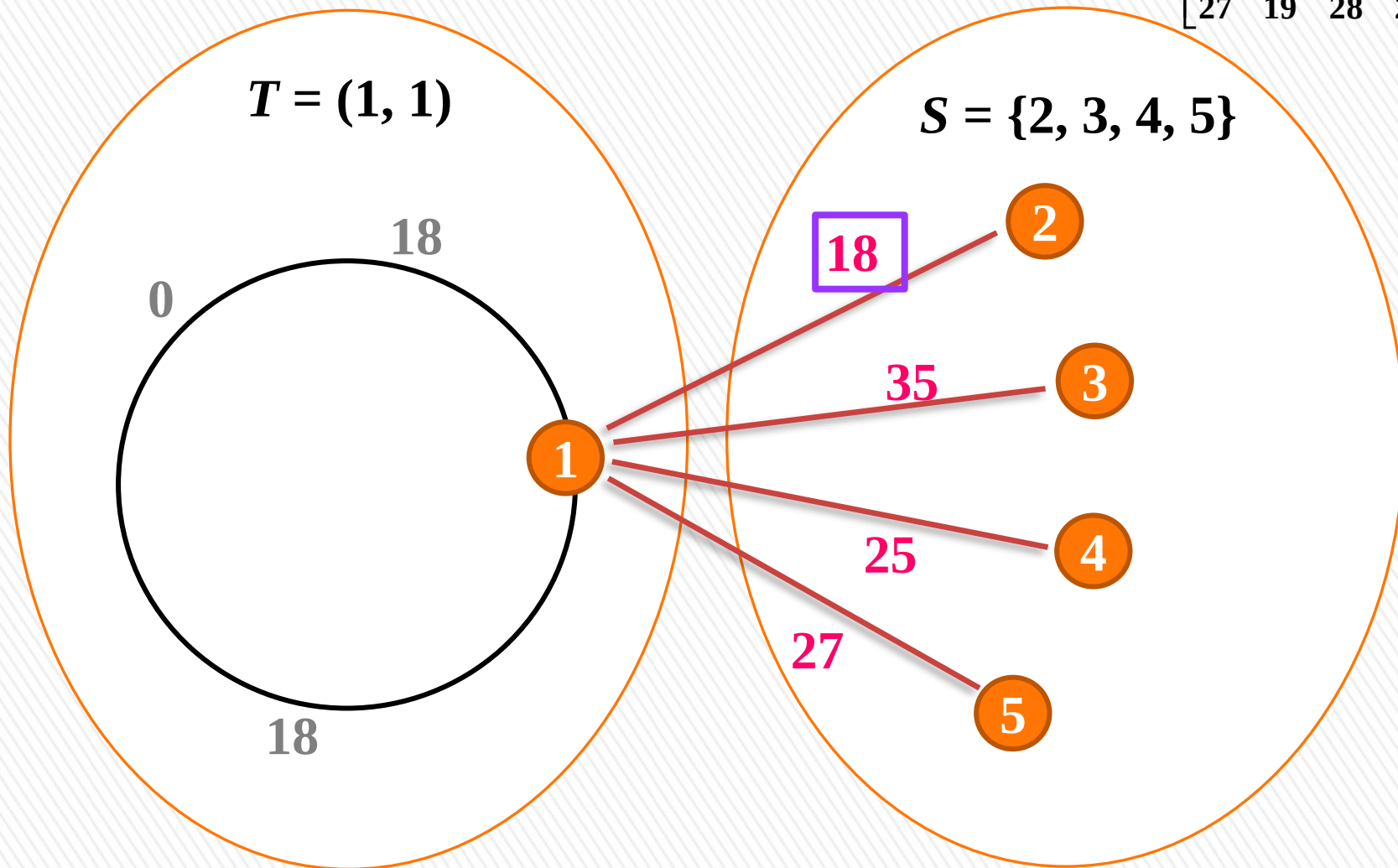
整体的最优解可通过一系列局部最优解达到. 每次的选择可以依赖以前作出的选择, 但不能依赖于后面的选择

最适用的问题:

最优子结构: 问题的整体最优解中包含着它的子问题的最优解。

“便宜”算法

0	18	35	25	27
18	0	23	21	19
35	23	0	17	28
25	21	17	0	24
27	19	28	24	0

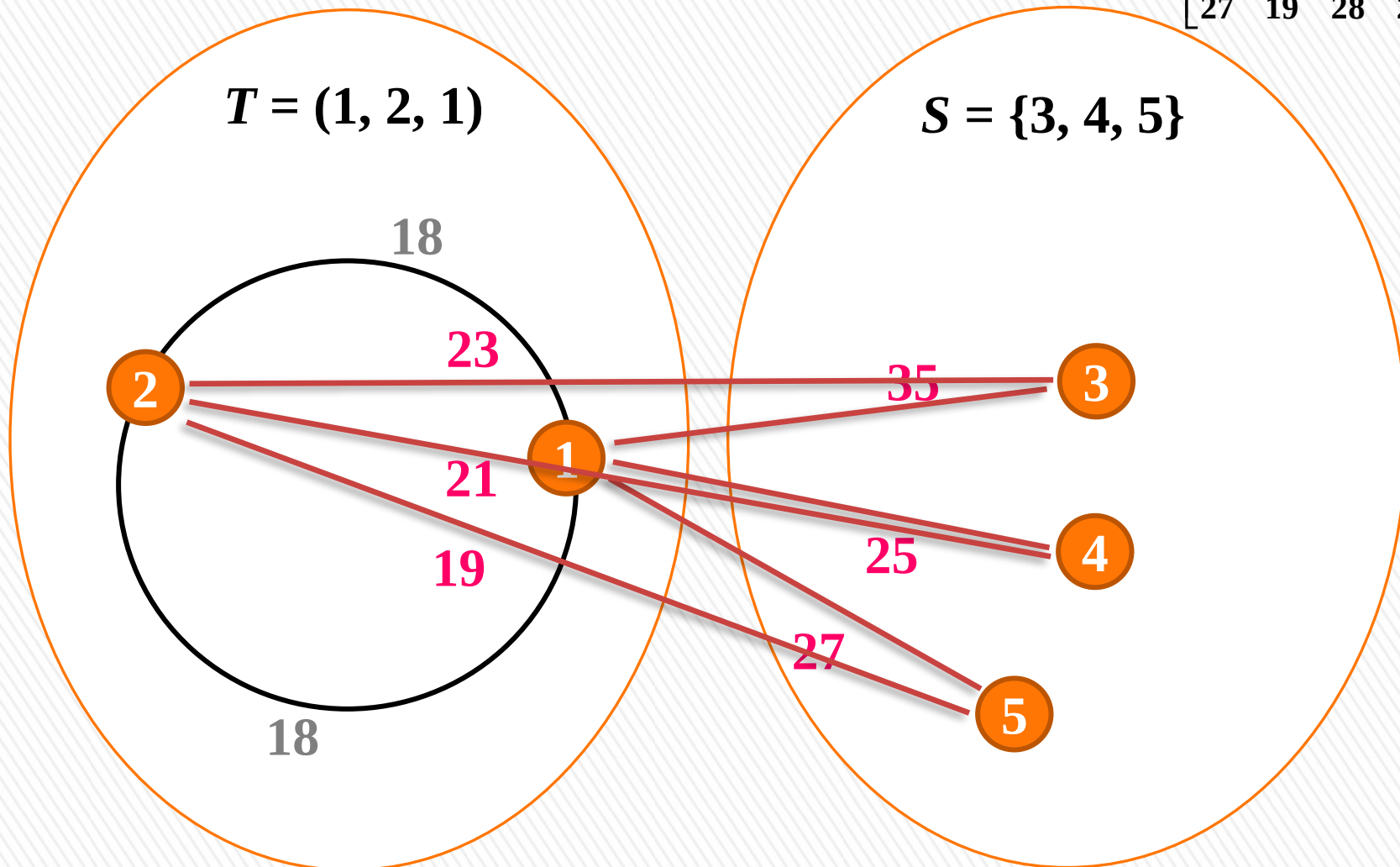


$$d = [0, 18, 35, 25, 27]$$

$$c = [1, 1, 1, 1, 1]$$

“便宜”算法

0	18	35	25	27
18	0	23	21	19
35	23	0	17	28
25	21	17	0	24
27	19	28	24	0



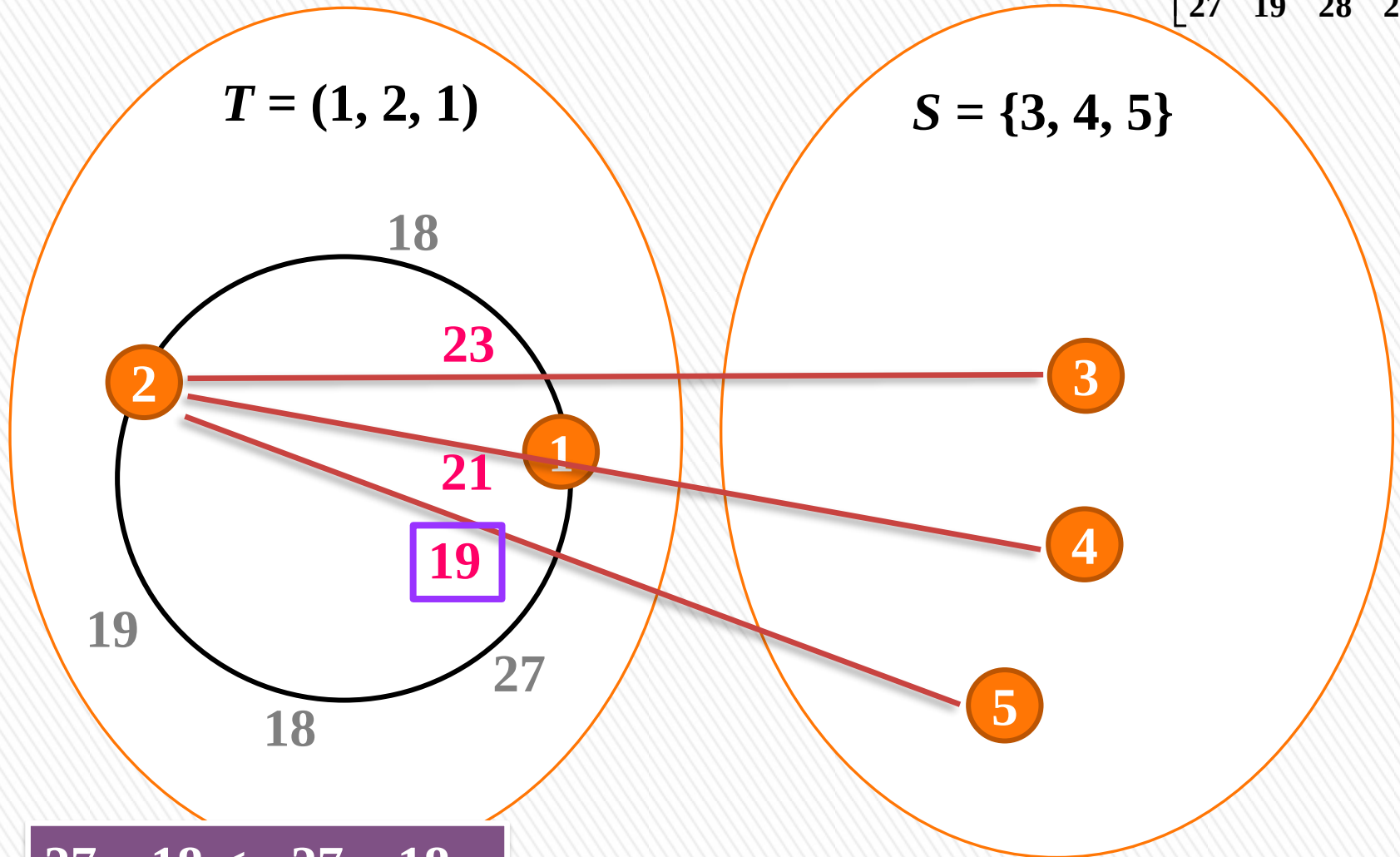
$d = [0, 18, 35, 25, 27]$

更新 $d = [0, 18, 23, 21, 19]$

$c = [1, 1, 2, 2, 2]$

“便宜”算法

0	18	35	25	27
18	0	23	21	19
35	23	0	17	28
25	21	17	0	24
27	19	28	24	0



$$27 - 18 \leq 27 - 18$$

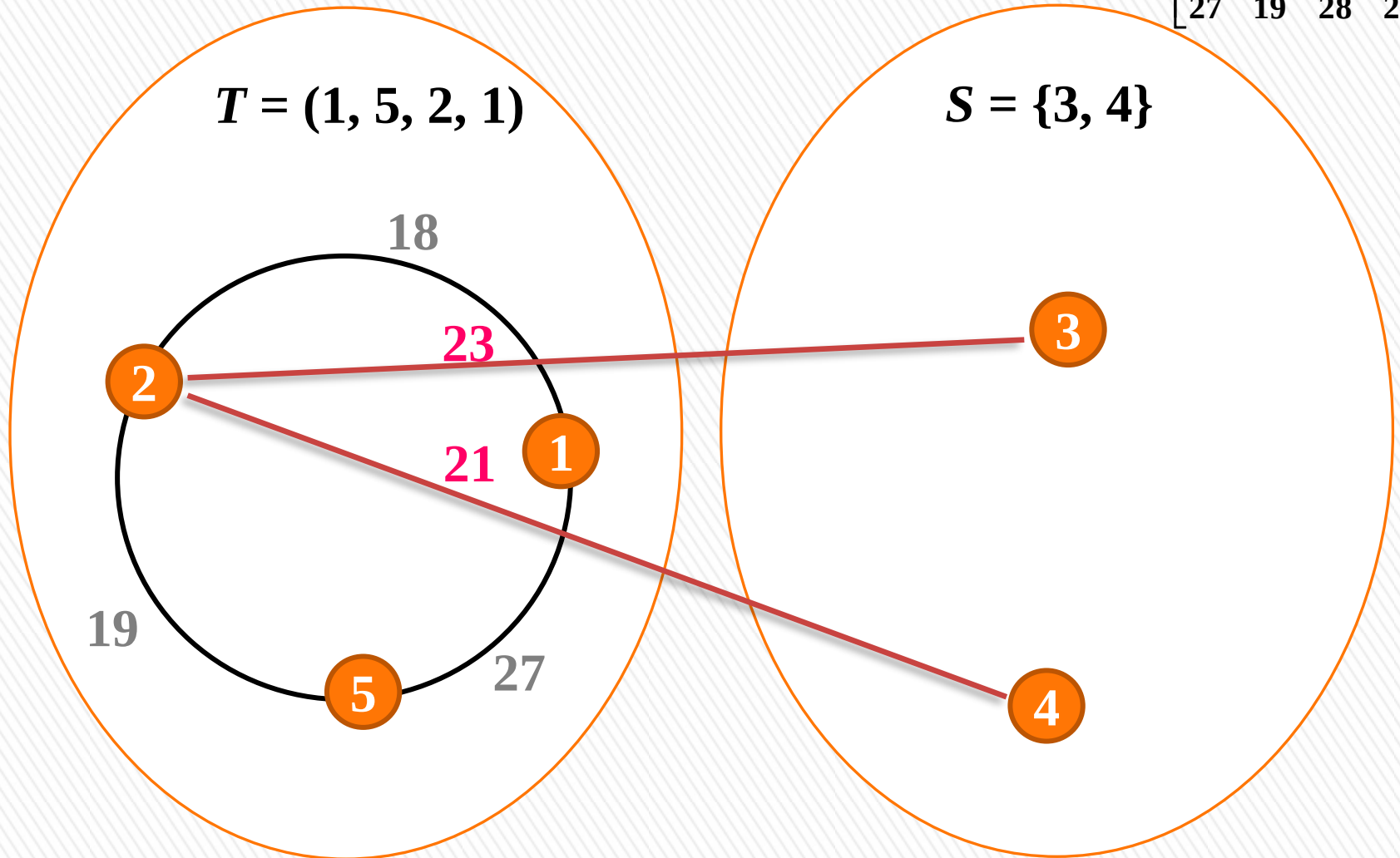
$$e_{21} - e_{51} \leq e_{21} - e_{51}$$

$$d = [0, 18, 23, 21, 19]$$

$$c = [1, 1, 2, 2, 2]$$

“便宜”算法

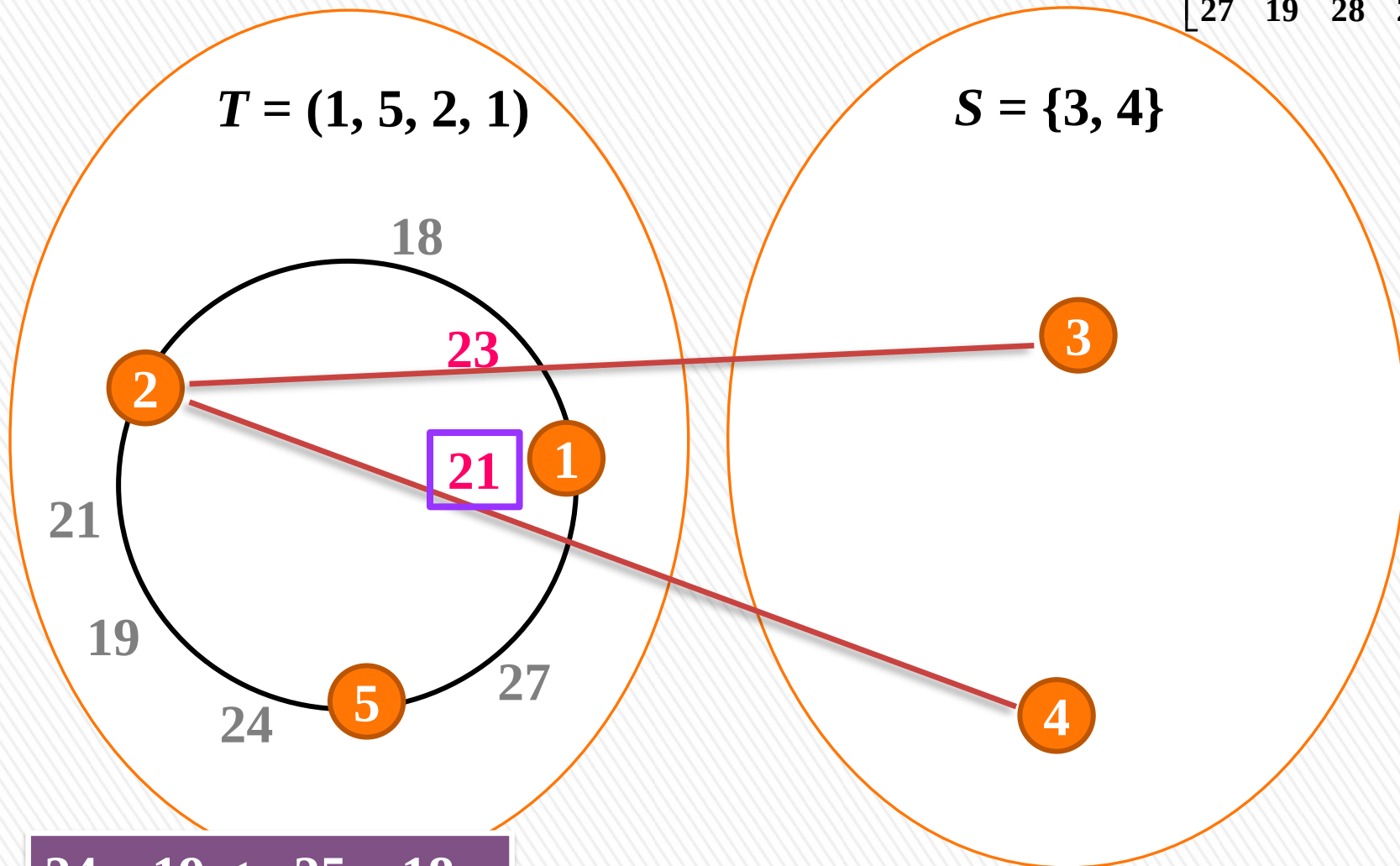
0	18	35	25	27
18	0	23	21	19
35	23	0	17	28
25	21	17	0	24
27	19	28	24	0



$$d = [0, 18, 23, 21, 19]$$

“便宜”算法

0	18	35	25	27
18	0	23	21	19
35	23	0	17	28
25	21	17	0	24
27	19	28	24	0



$$24 - 19 \leq 25 - 18$$

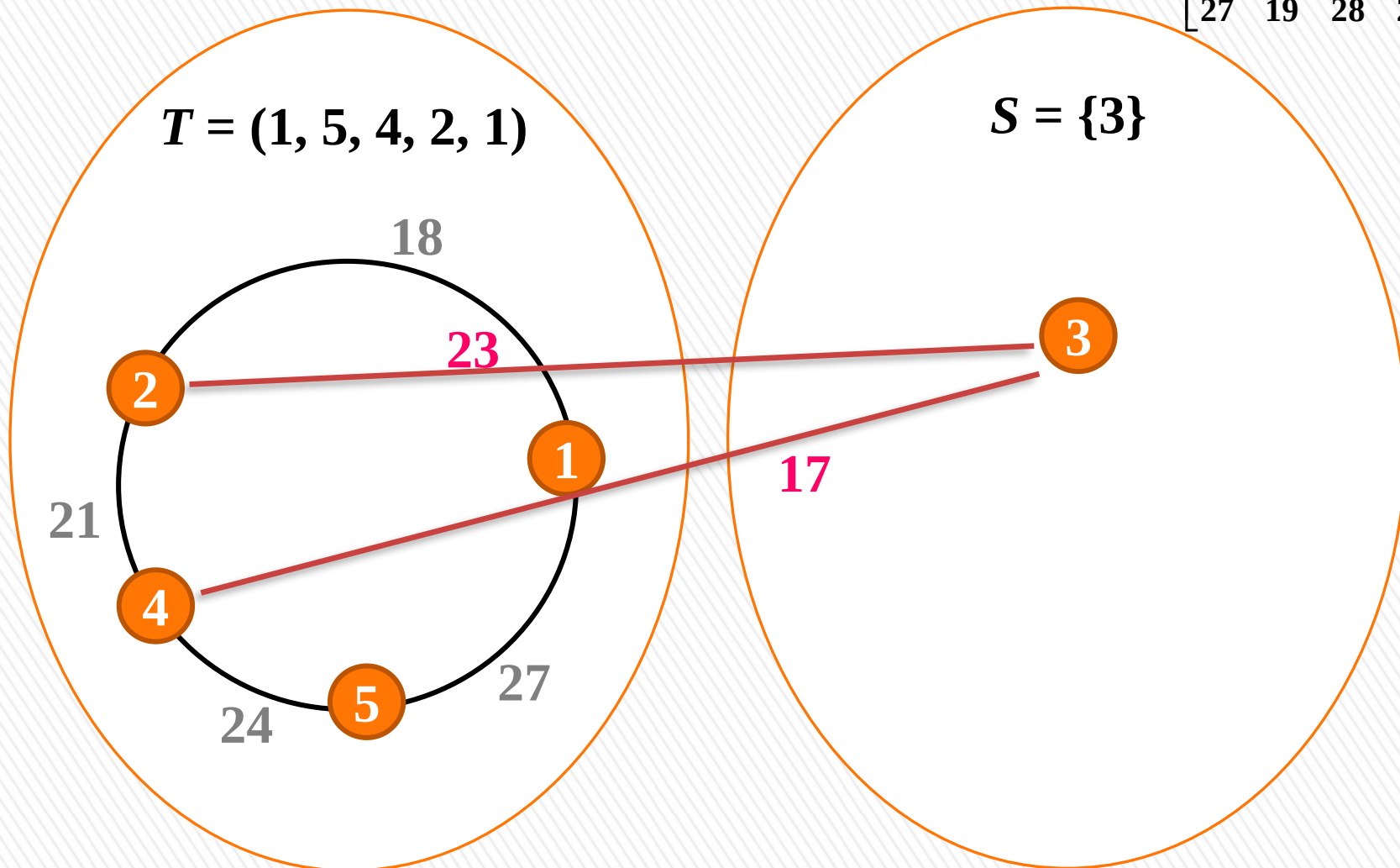
$$e_{45} - e_{25} \leq e_{41} - e_{21}$$

$$d = [0, 18, 23, 21, 19]$$

$$c = [1, 1, 2, 2, 2]$$

“便宜”算法

0	18	35	25	27
18	0	23	21	19
35	23	0	17	28
25	21	17	0	24
27	19	28	24	0

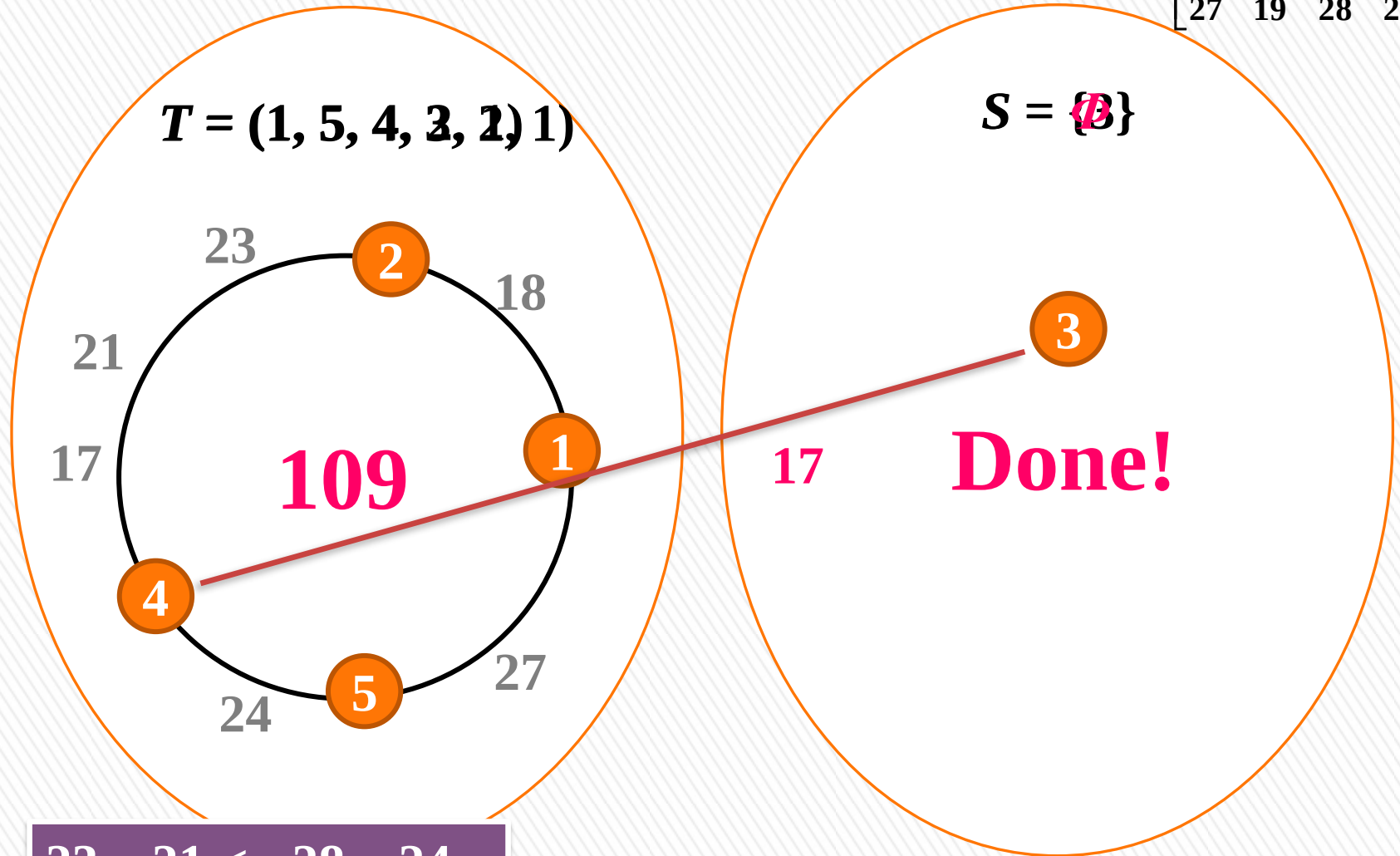


$$d = [0, 18, 23, 21, 19]$$

$$c = [1, 1, 4, 2, 2]$$

“便宜”算法

0	18	35	25	27
18	0	23	21	19
35	23	0	17	28
25	21	17	0	24
27	19	28	24	0



$$23 - 21 \leq 28 - 24$$

$$e_{32} - e_{42} \leq e_{35} - e_{45}$$

$$d = [0, 18, 17, 21, 19]$$

$$c = [1, 1, 4, 2, 2]$$

“便宜”算法

■ “便宜”算法过程

- T 是一个不断扩充的初级回路
- 最初 T 是一个自环
- 算法找一个与 T 最近的节点 j ，将 j 插入 T 中
- 假设与 T 中的 t 最近，具体插入 t 的前面或后面，依据 j 插入后回路 T 长度增量的大小而定
如果 $w(j, t) + w(j, t_1) - w(t, t_1) \leq w(j, t) + w(j, t_2) - w(t, t_2)$ ，
则 j 插到 t 与 t_1 之间；否则 j 在 t 与 t_2 之间

“便宜”算法

a. 设 $w(i, j)$ 为边权值，对任意 i ， $w(i, i) = 0$ ；

置 $S = \{2, 3, \dots, n\}$ ，序列 $T = (1, 1)$ 。

b. 在 S 中，令 $(j, t) = \operatorname{argmin}_{i \in S, k \in T} w(i, k)$ ，即： $j \in S, t \in T$

对回路 T 中的边 $(t, t_1), (t, t_2)$ ，

如果 $w(j, t) + w(j, t_1) - w(t, t_1) \leq w(j, t) + w(j, t_2) - w(t, t_2)$ ，

则 j 插到 t 与 t_1 之间，否则 j 在 t 与 t_2 之间；

令 $S = S - \{j\}$ ，若此时 $S = \emptyset$ 则结束，否则转(b)

朴素的实现：计算复杂度为 $O(n^3)$

借助辅助数组 d ，可以在 $O(n)$ 时间计算 argmin ：

此时“便宜”算法的计算复杂度为 $O(n^2)$

“便宜”算法

- 分支定界法与“便宜”算法的比较
 - “便宜”算法的效率大大优于分支与界法 ($O(n^2)$ vs. $O(n!)$)
 - 分支与界法的质量优于“便宜”算法
精确最优解 vs. 近似最优解
便宜值 T 与最优值 Q 相比 只能保证 $T/Q < 2$
 - 但实际中“便宜”算法与最优解非常接近

最短路径问题

最短路径

- 最短路问题是网络理论中应用最广的问题之一。许多优化问题可以使用这个模型，如设备更新、管道铺设、线路安排、厂区布局等。
- 最短路问题的一般提法如下：设 $G = (V, E)$ 为连通图，图中各边 v_i, v_j 有权 w_{ij} ($w_{ij} = \infty$ 表示 v_i, v_j 间无边)

v_s, v_t 为图中任意两点，求一条道路 μ ，使它是从 v_s 到 v_t 的所有道路中总权最小的道路。即：

$$L(\mu) = \sum_{(v_i, v_j) \in \mu} w_{ij}$$

最小.

最短路径模型

■ 最短路径可研究的情况包括

	边权为 正数	边权为 1	边权为 任意实数
某点到其他点 (指定的两点)	?		
任意两点之间			

讨论**权值为正**的情况下某点到其他各个节点最短路径**的问题**

最短路径

- 正权图中两点间的最短路径一定是**初级道路**。
- 路径长度定义 v_1 到 v_i 的一条路径 $P(i)$ 长度记为 $\pi(i)$

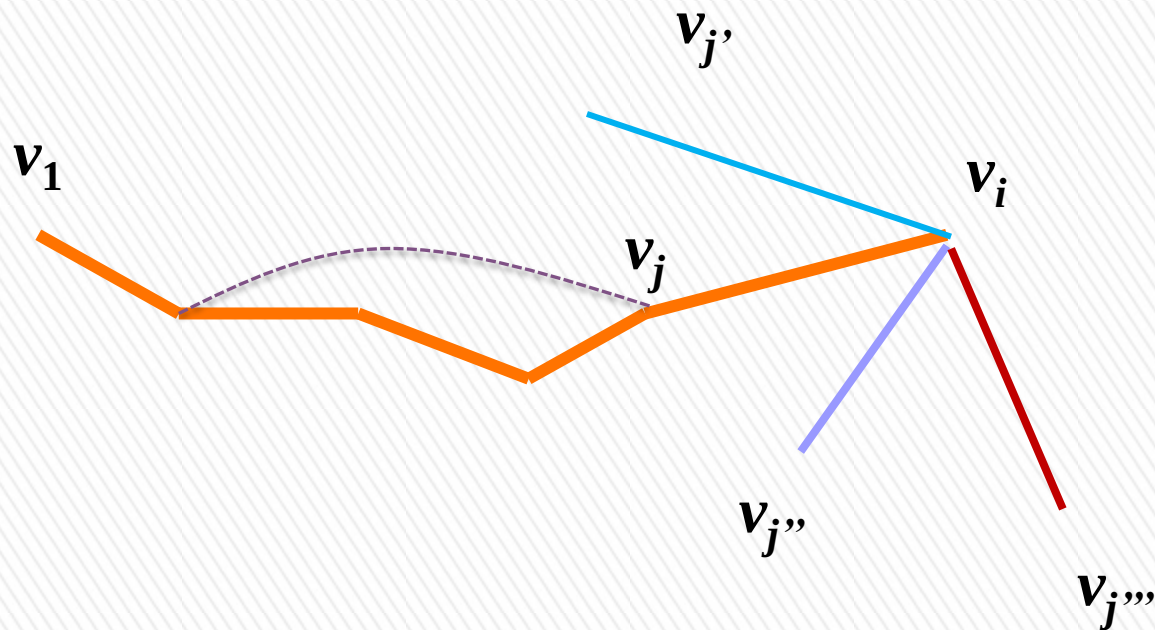
$$\pi(i) = \sum_{e \in P(i)} w(e)$$

- **引理2.6.1** 正权图中任一条最短路径长度大于其局部路径长度。



最短路径

- **引理2.6.2** 正权图中若 $P(i)$ 是 v_1 到 v_i 的最短路，且 $v_j \in P(i)$ ，则 $P(j)$ 也是 v_1 到 v_j 最短路。



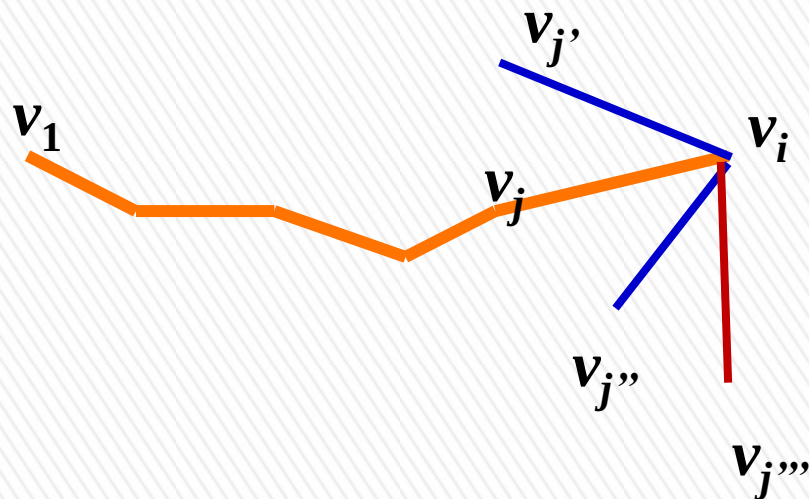
最短路径

假定已经知道从 v_1 到其它各点的最短路 $P(i_k)$ ($k = 1, 2, \dots, n$), 且满足

$$\pi(1) = \pi(i_1) \leq \pi(i_2) \leq \dots \leq \pi(i_n)$$

如果 $k > l$ ($l \geq 1$), 则 $P(i_k)$ 不可能是 $P(i_l)$ 的一部分。

$$\pi(i_l) = \min_{1 \leq j < l, j \in \Gamma^{-} i_l} (\pi(i_j) + w_{ij} i_l)$$



Dijkstra算法



■ 基本思想 1959提出

按路径长度递增的次序产生最短路径

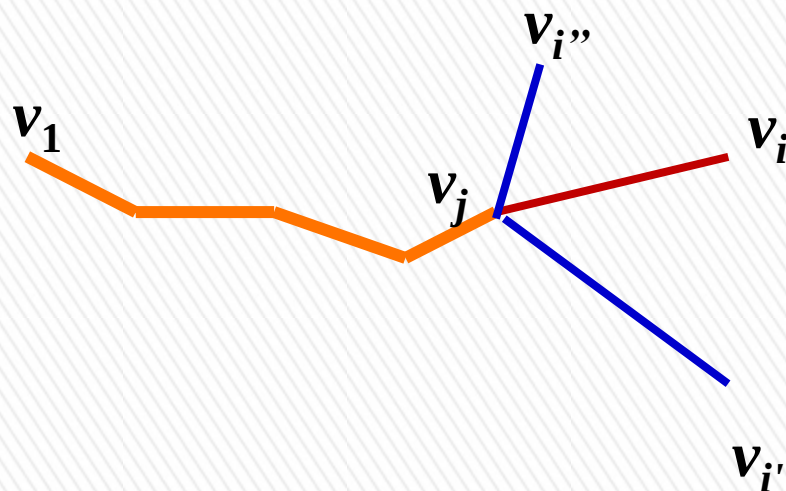
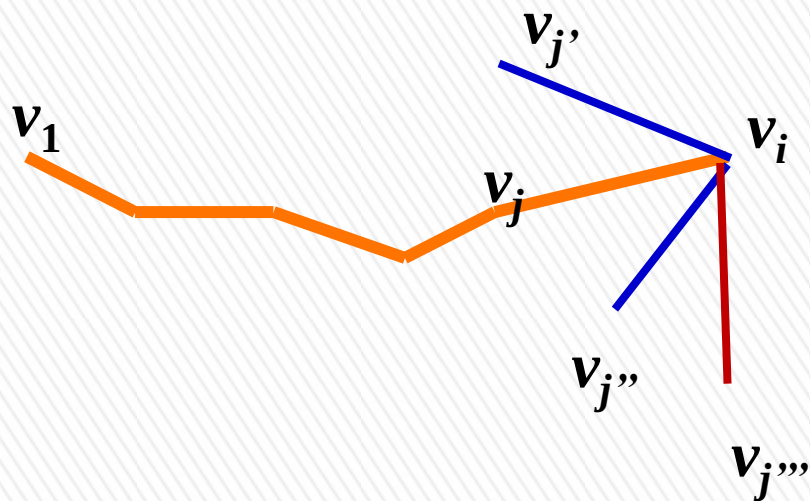
- 设置一个集合 P ，存放已**求出最短路径的顶点**
- $T = V - P$ 是尚未确定最短路径的顶点集合；
- 每个顶点对应一个**距离值**（用 Π 表示）
 - + 集合 P 中顶点的距离值：从顶点 v_1 到该顶点的最短路径长度；
 - + 集合 T 中顶点的距离值：从顶点 v_1 到该顶点的只包括集合 P 中顶点为中间顶点的最短路径长度。
- **Idea:** T 里面最小元素一定是最短路，可以将其转移到 P 中，**同时更新 T 中其他元素的距离**；
- 如何实现：分步完成以上过程

Dijkstra算法

■ 如何“更新 T 中其他元素的距离”？

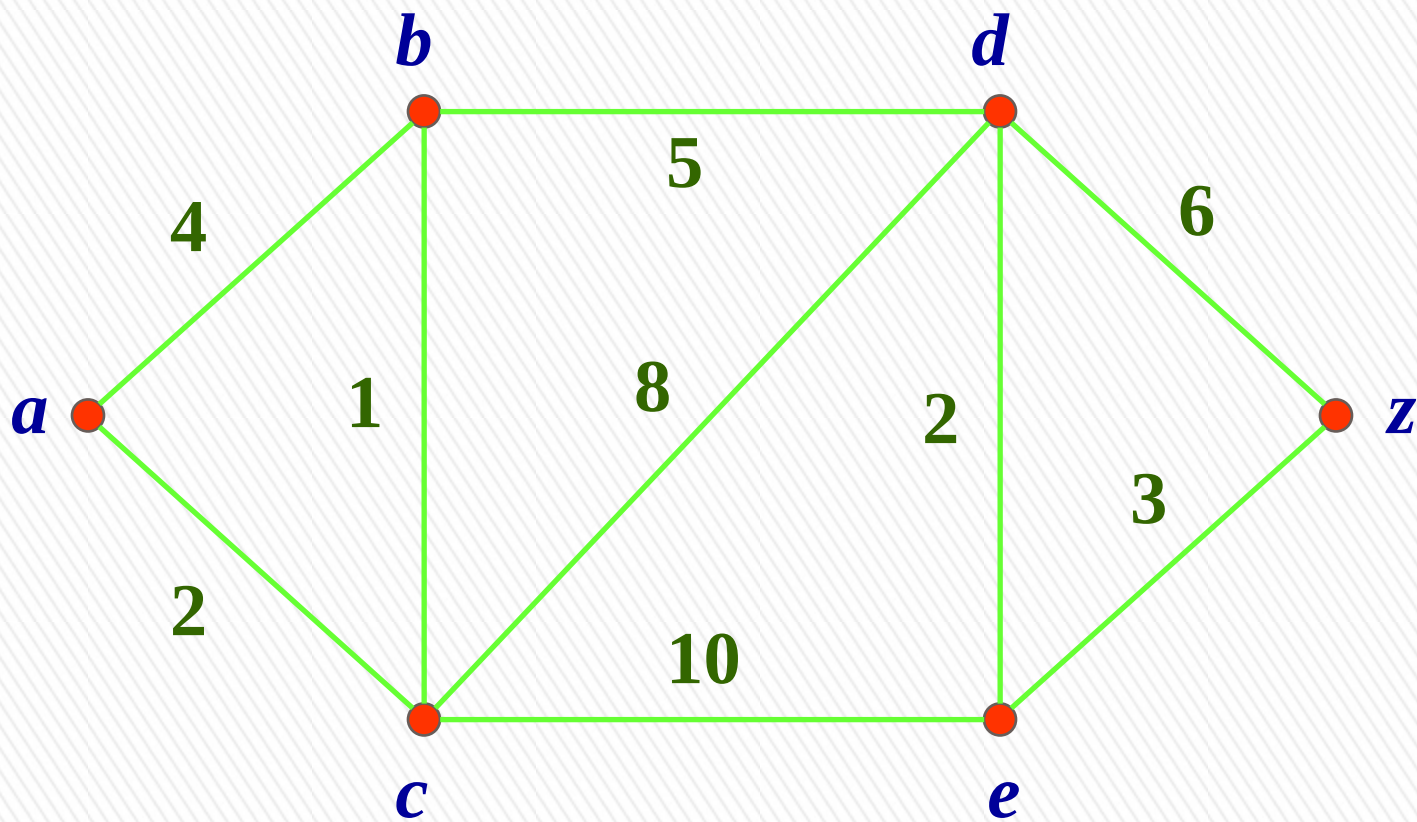
- 用 π 表示记录的从 v_1 出发的距离值
- 对于当前 T 中 π 最小的节点 v ，检查其每一个后继 u ，并更新其距离值
- $\Pi(u) = \min\{\Pi(u), \Pi(v) + w(v,u)\}$

■ 看前驱 v.s 看后继 $\pi(i_l) = \min_{1 \leq j < l, j \in \Gamma^{-i_l}} (\pi(i_j) + w_{i_j i_l})$



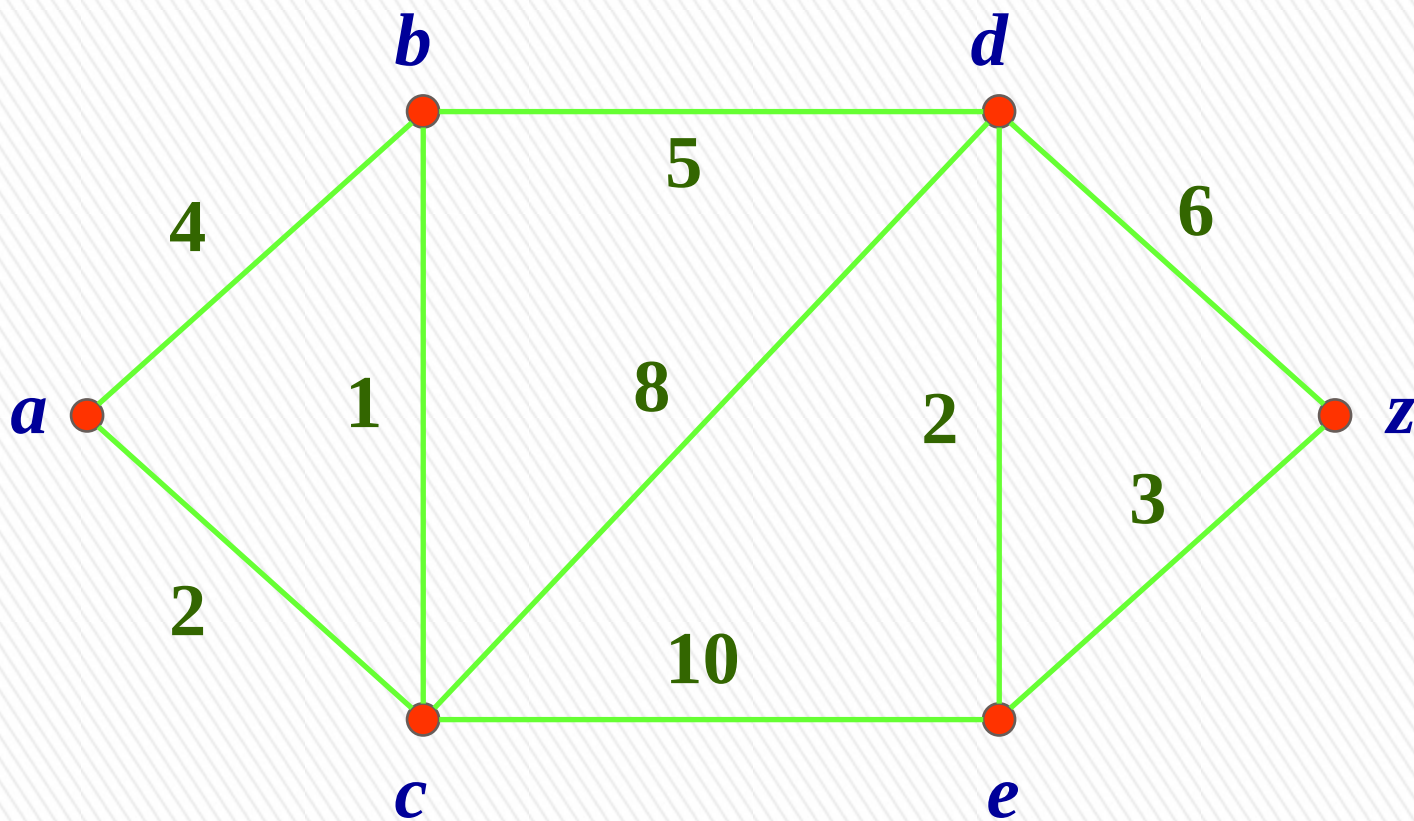
Dijkstra实例

例：求下图最短路径



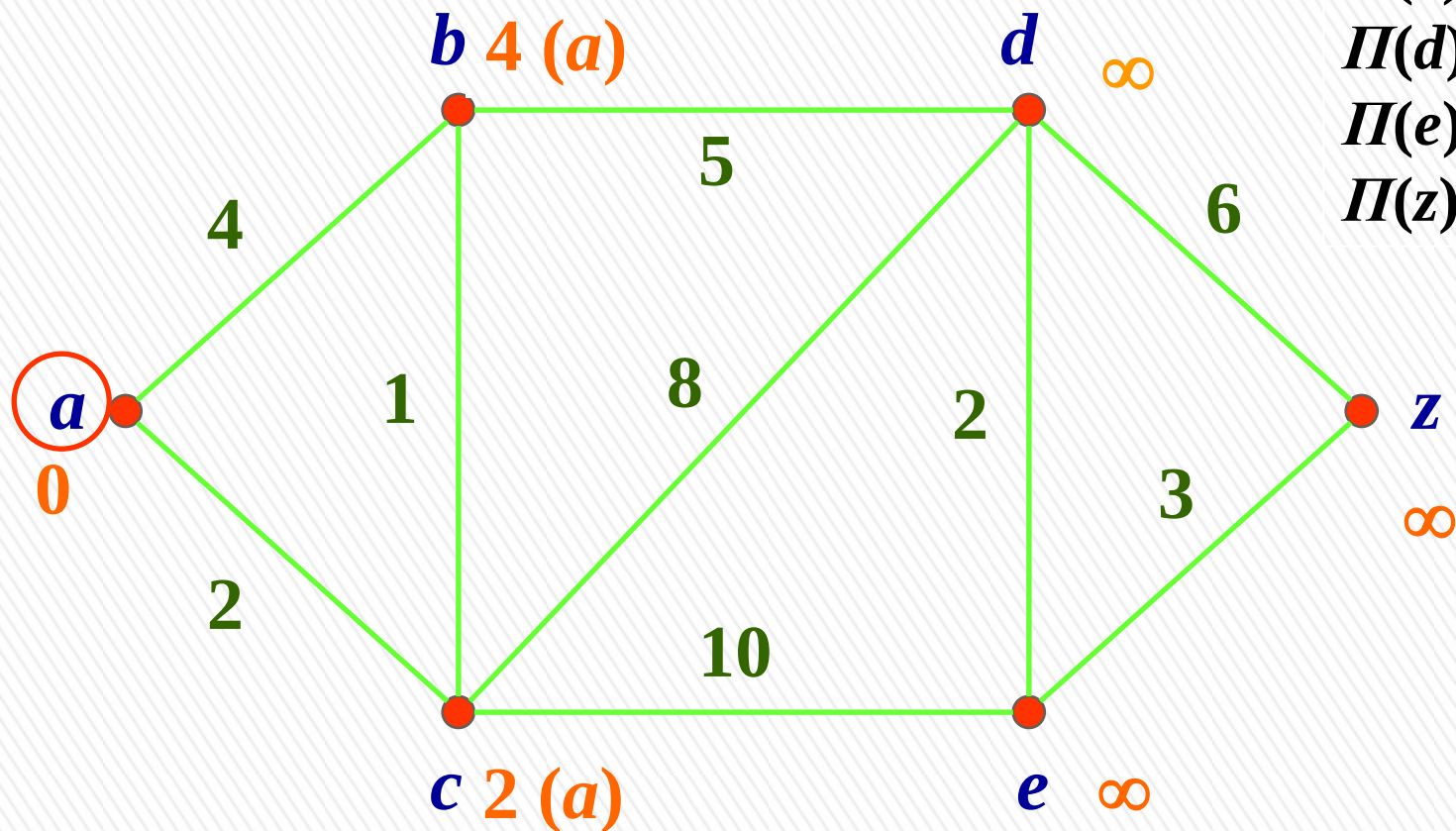
Dijkstra实例

解：步骤1



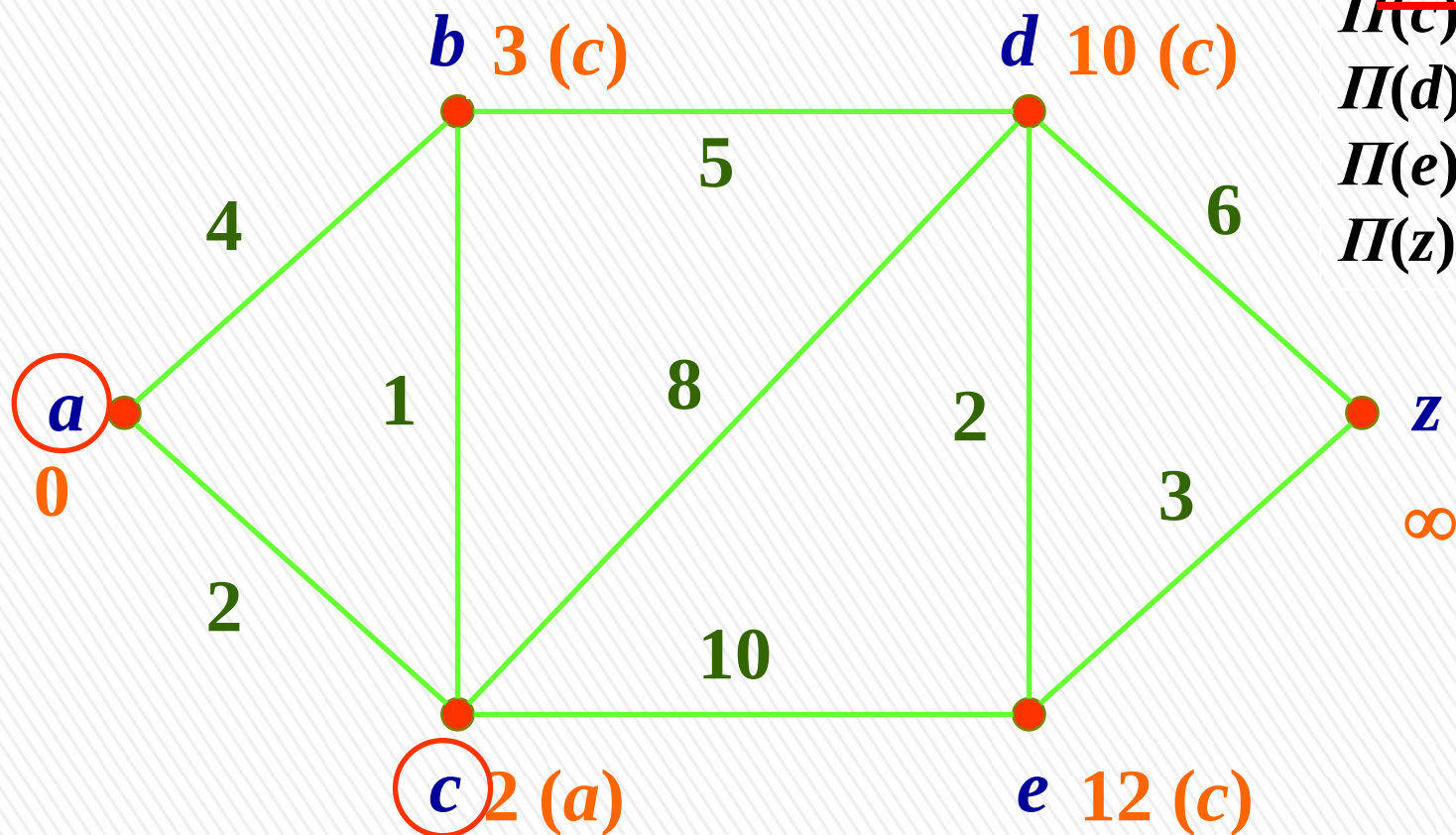
Dijkstra实例

解：步骤1



Dijkstra实例

解：步骤2



$$\min \Pi = 2$$

$$\Pi(b) = 4 \rightarrow 3$$

~~$$\Pi(c) = 2$$~~

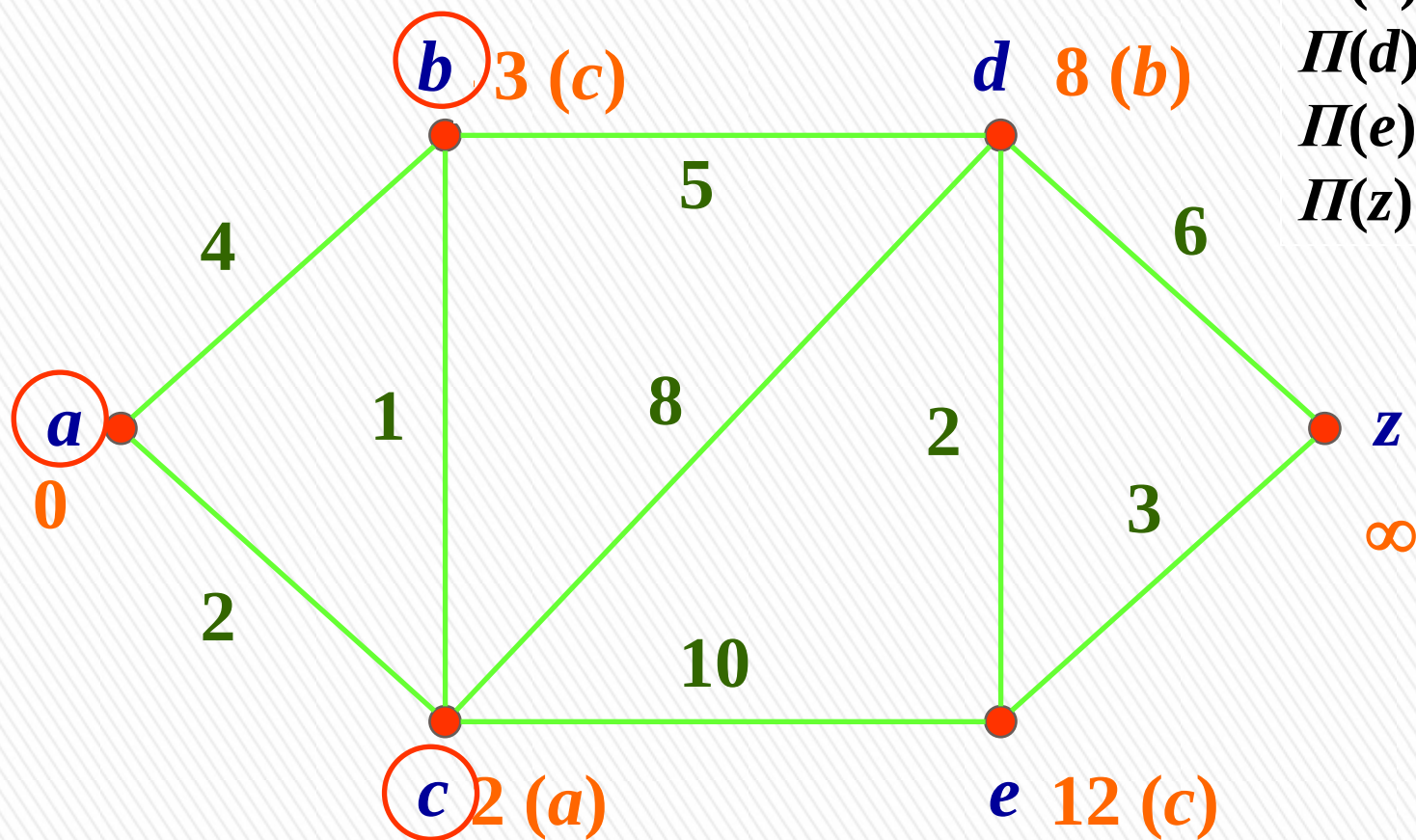
$$\Pi(d) = \infty \rightarrow 10$$

$$\Pi(e) = \infty \rightarrow 12$$

$$\Pi(z) = \infty$$

Dijkstra实例

解：步骤3



$\min \Pi = 3$

~~$\Pi(b) = 3$~~

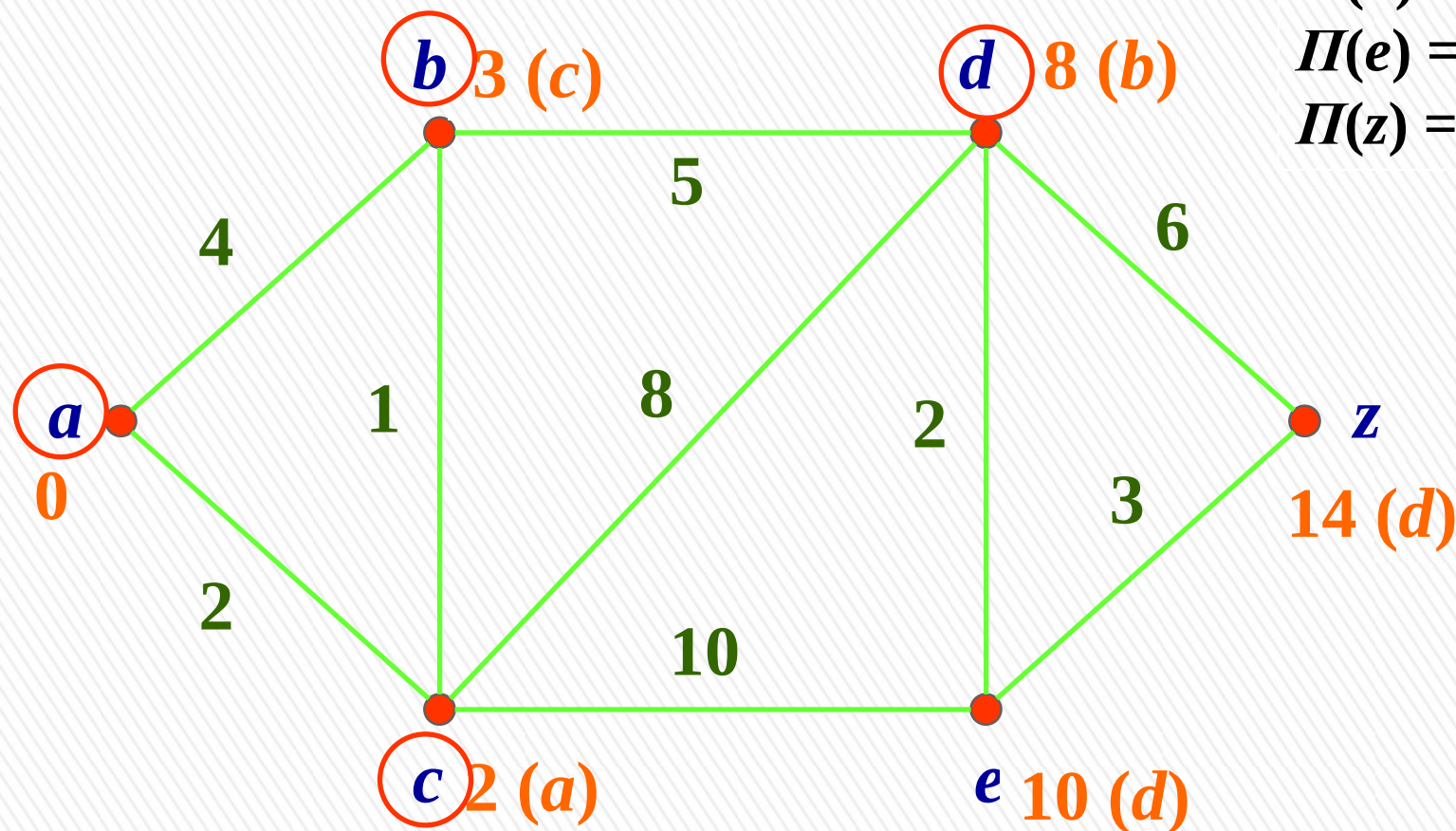
$\Pi(d) = 10 \rightarrow 8$

$\Pi(e) = 12$

$\Pi(z) = \infty$

Dijkstra实例

解：步骤4



$\min \Pi = 8$

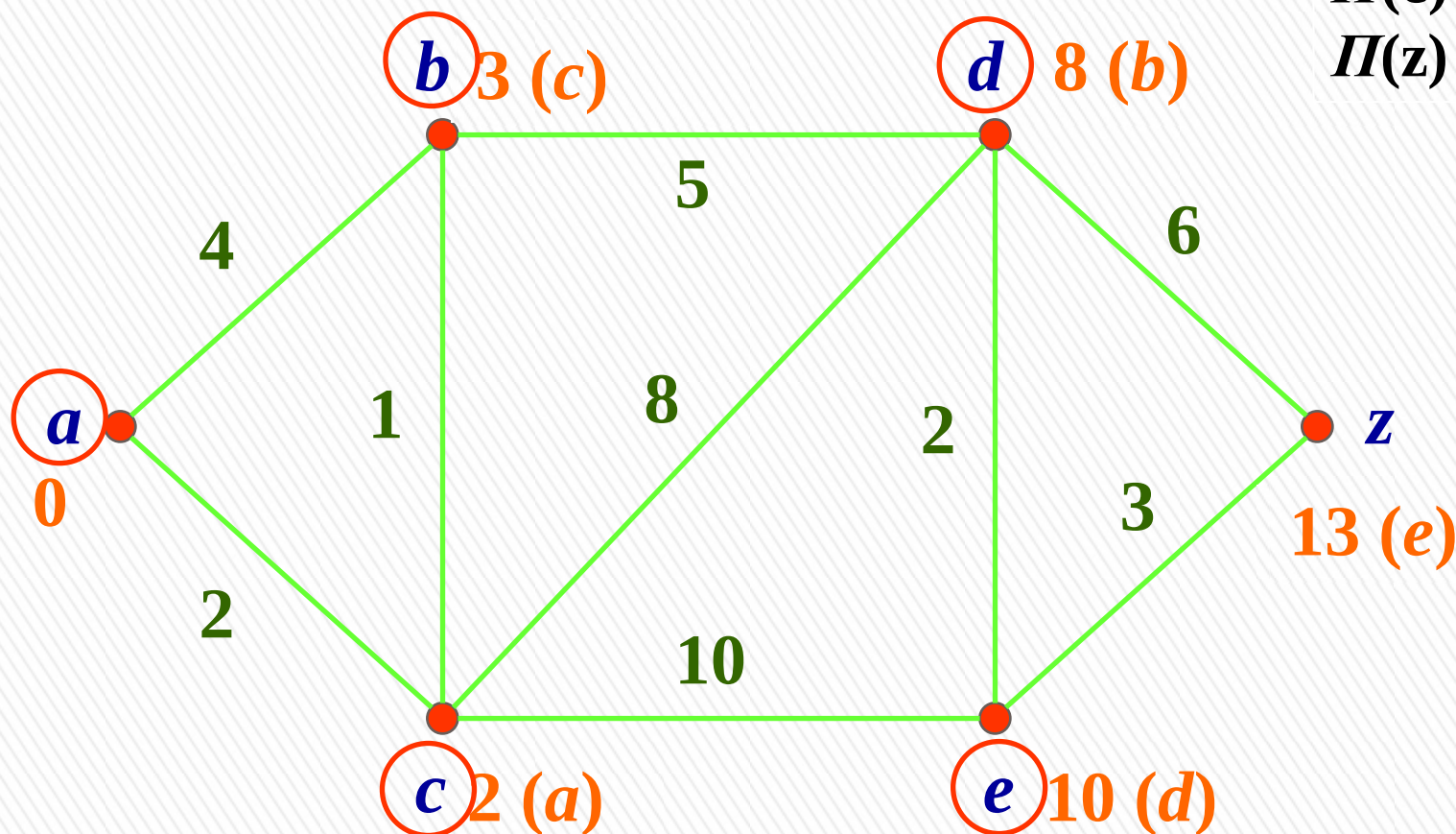
~~$\Pi(d) = 8$~~

$\Pi(e) = 12 \rightarrow 10$

$\Pi(z) = \infty \rightarrow 14$

Dijkstra实例

解：步骤5



$\min \Pi = 10$

~~$\Pi(e) = 10$~~

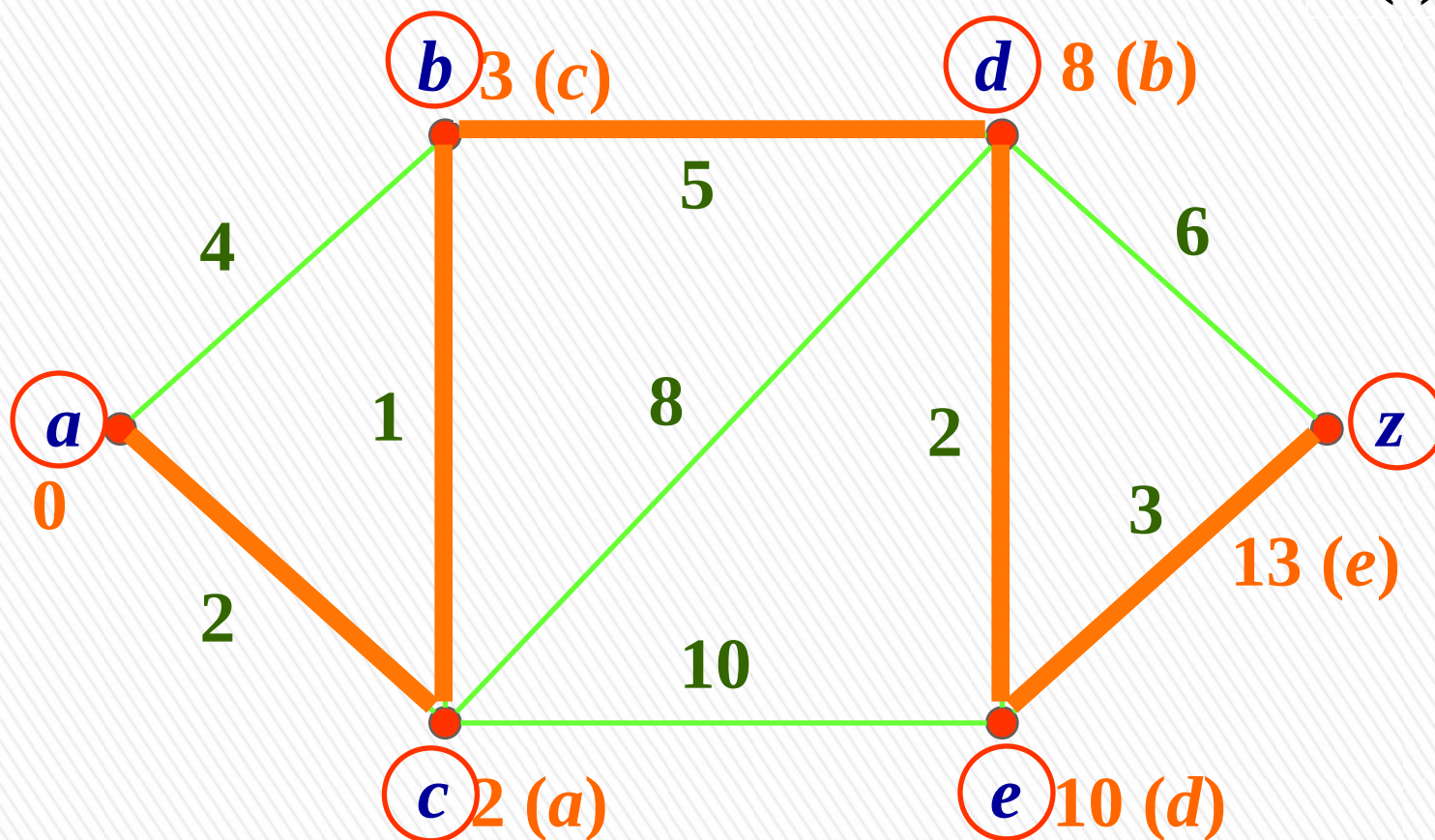
$\Pi(z) = 14 \rightarrow 13$

Dijkstra实例

解：步骤6

$\min \Pi = 13$

~~$\Pi(z) = 13$~~



Dijkstra算法

■ 贪心算法

将问题的求解过程看作是一系列选择, 每次选择一个输入, 每次选择都是当前状态下的最好选择(局部最优解). 每作一次选择后, 所求问题会简化为一个规模更小的子问题. 从而通过每一步的最优解(希望)逐步达到整体的最优解。

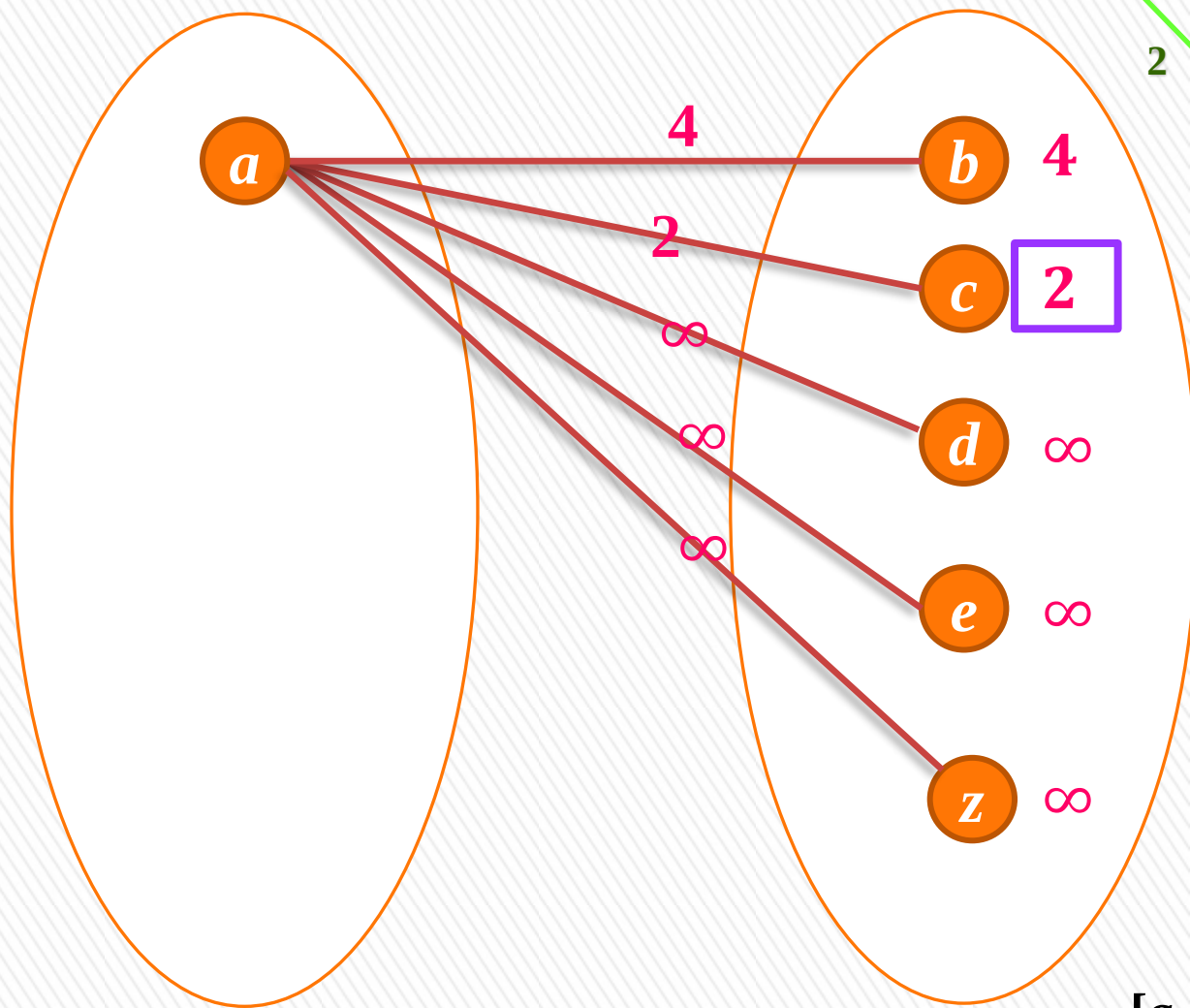
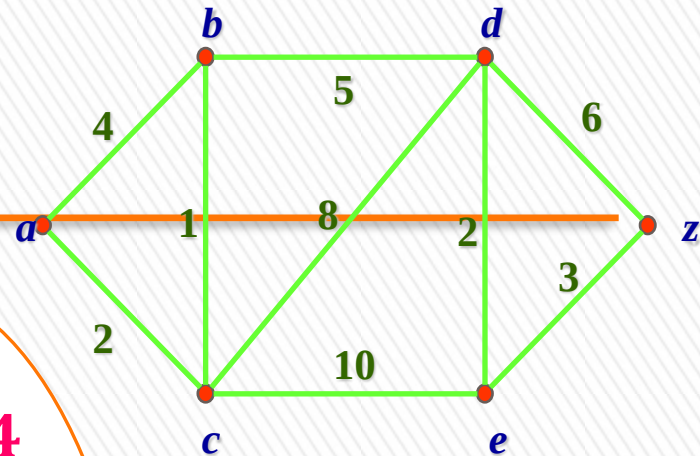
达到最优解的前提:

整体的最优解可通过一系列局部最优解达到. 每次的选择可以依赖以前作出的选择, 但不能依赖于后面的选择

最适用的问题:

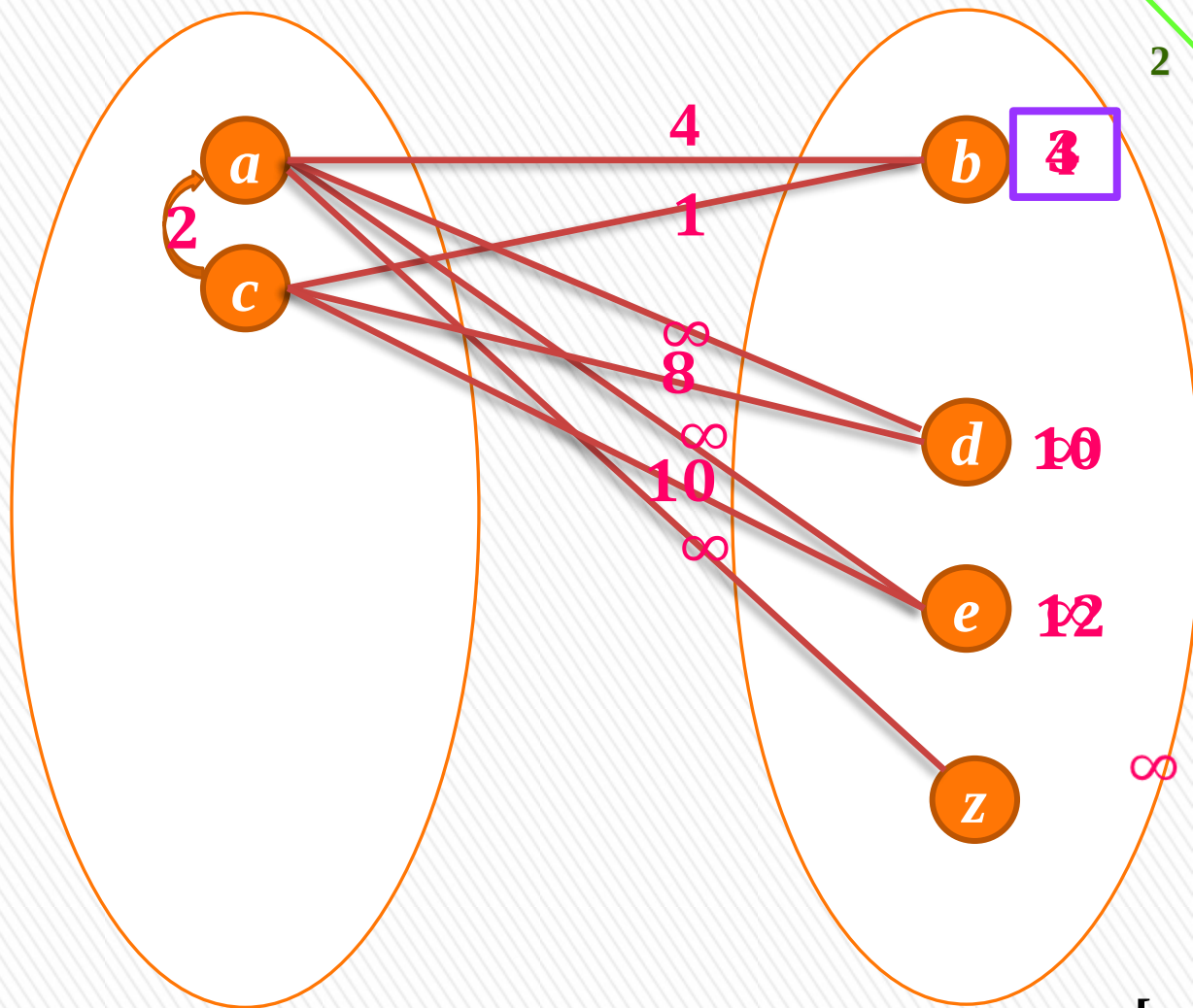
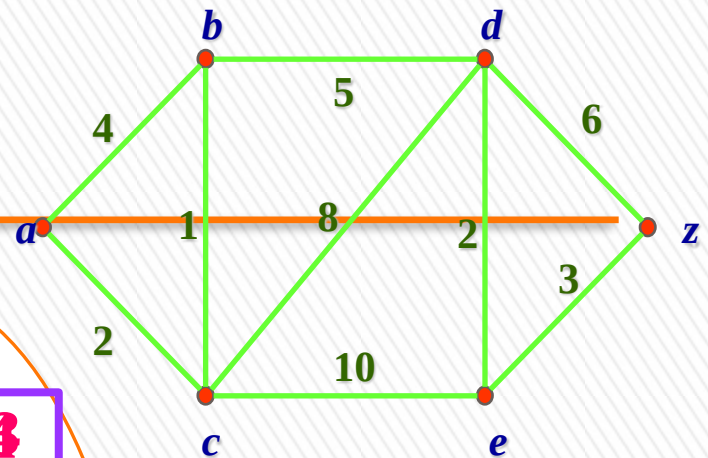
最优子结构: 问题的整体最优解中包含着它的子问题的最优解。

Dijkstra算法



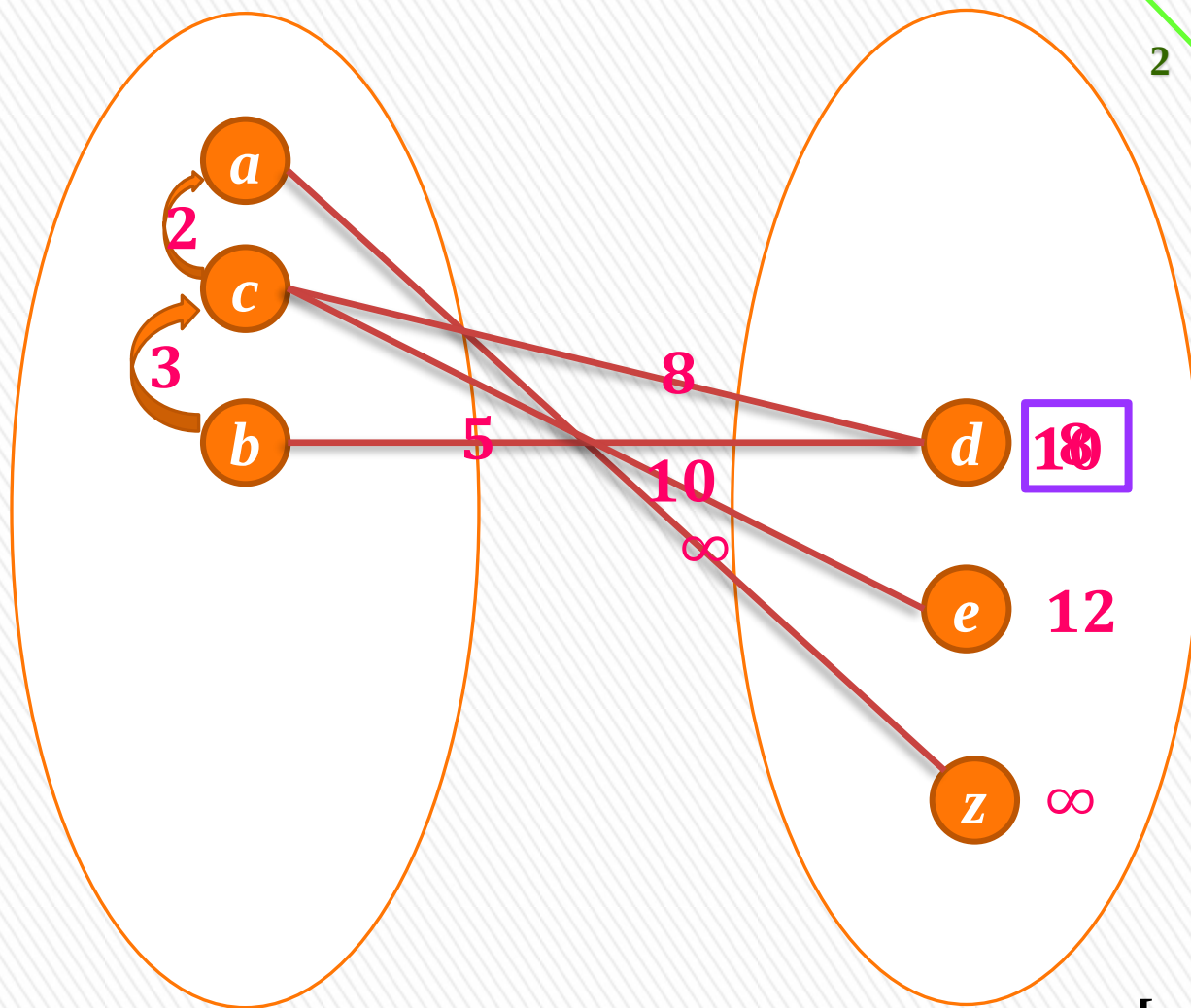
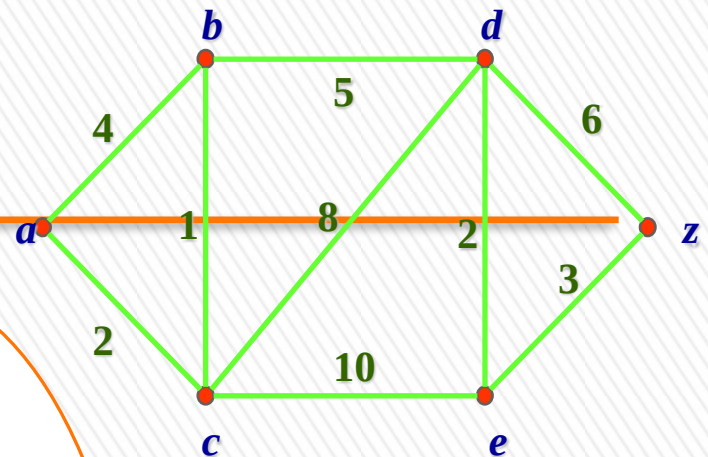
$[a, b, c, d, e, z]$
 $c = [a, a, a, a, a, a]$

Dijkstra算法



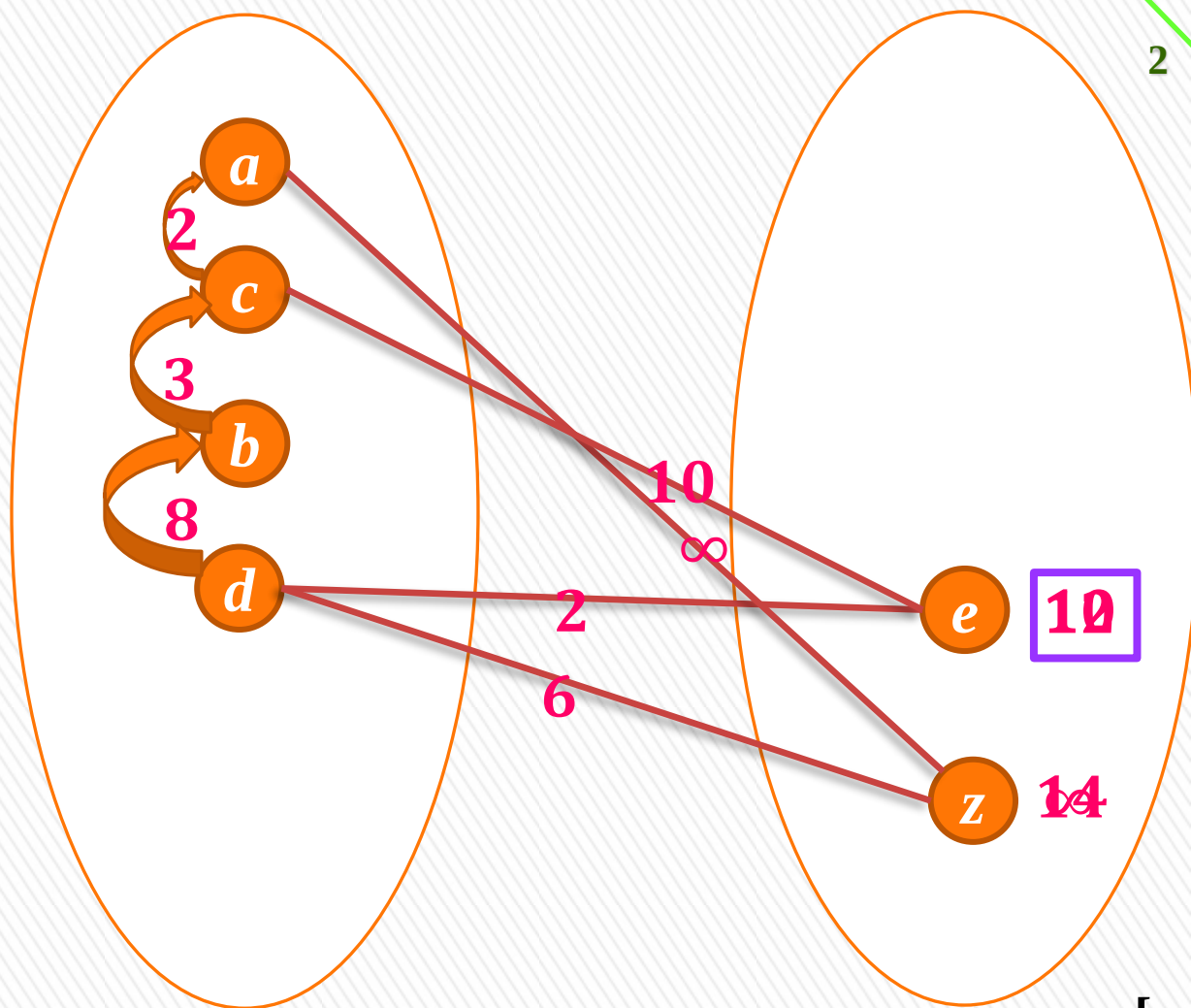
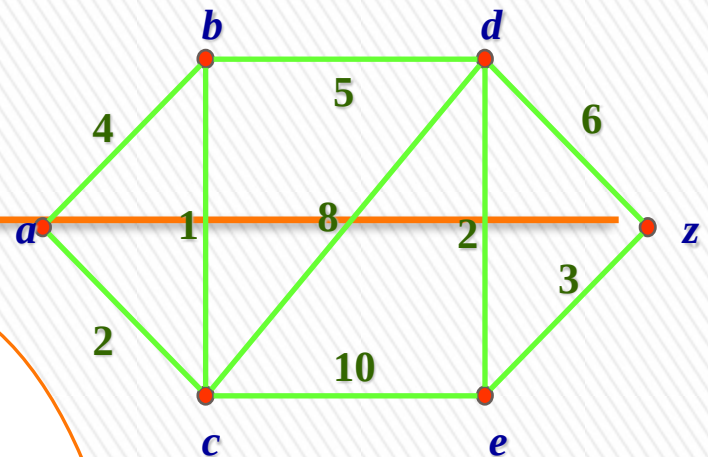
$[a, b, c, d, e, z]$
 $c = [a, a, a, a, a, a]$

Dijkstra算法



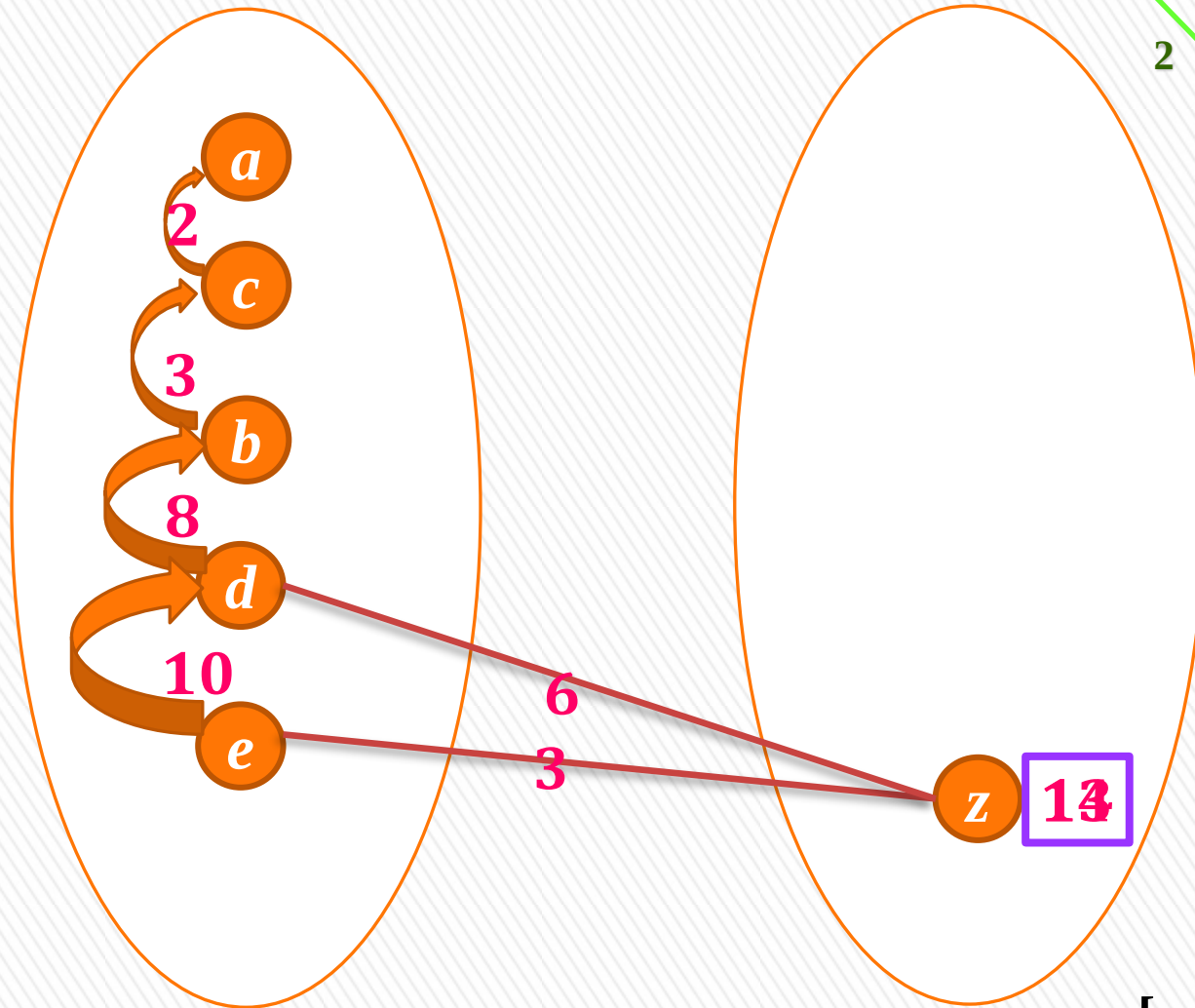
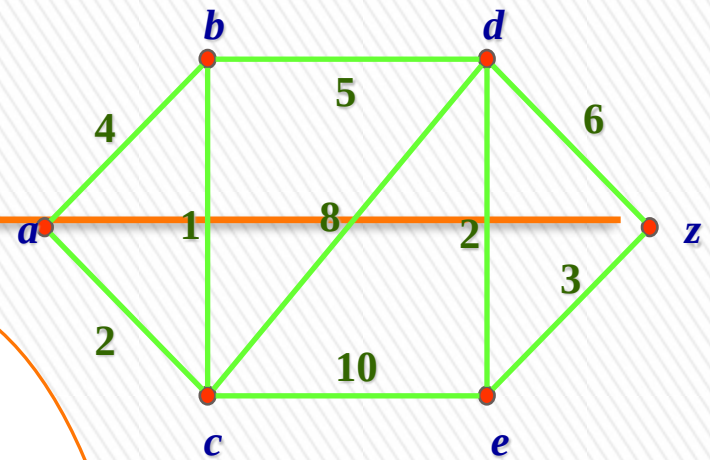
$[a, b, c, d, e, z]$
 $c = [a, c, a, \mathbf{b}, a, a]$

Dijkstra算法



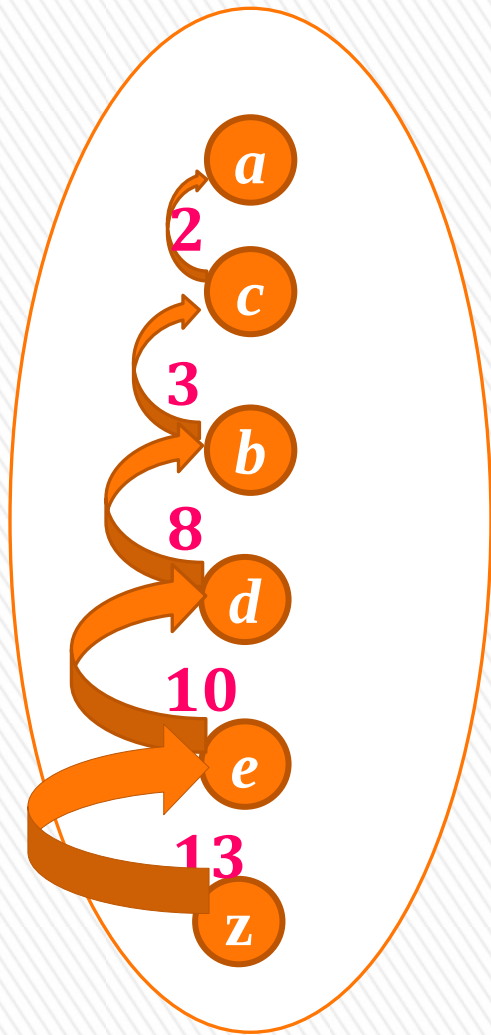
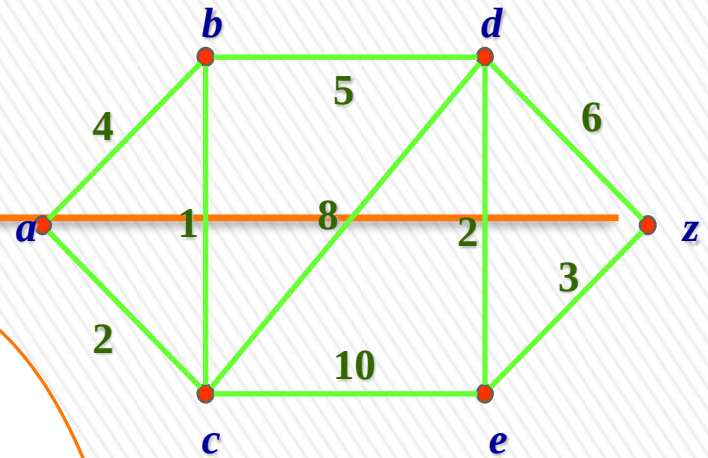
$c = [a, b, c, d, e, z]$
 $c = [a, c, a, b, d, d]$

Dijkstra算法



$[a, b, c, d, e, z]$
 $c = [a, c, a, b, d, \textcolor{red}{e}]$

Dijkstra算法



$[a, b, c, d, e, z]$
 $c = [a, c, a, b, d, e]$

Dijkstra算法

■ 算法步骤

算法复杂度 $O(n^2)$

- a. 初始化:** 集合 P 中只有顶点 v_1 , $\Pi(v_1)$ 为0, 集合 $T = V - P$ 中顶点 v_i 的距离值为边 (v_1, v_i) ($i=2, \dots, n$)的权 w_{1i} , 如果 v_1 和 v_i 间无边直接相连, 则 $\Pi(i)$ 为 ∞ ;
 - b. 贪心选择:** 在集合 T 中选择距离值 π 最小的顶点 j 加入集合 P , $P = P + \{j\}$, $T = T - \{j\}$;
 - c. 修正:** 对集合 T 中 j 的后继节点 i 的距离值进行修正: 如果加入顶点 j 后, 使 v_1 到 v_i 的距离值比原来的距离值更小, 则修改 v_i 的距离值 $\Pi(i) = \min(\Pi(i), \Pi(j) + w_{ji})$;
- 重复**b**和**c**, 直到 T 为空为止。

Dijkstra算法

■ 算法步骤

存储最短路径:

- **初始化**: 增设一个 n 维向量 Q , 初值为1
- **修正**: 对集合 T 中各顶点的距离值进行修正: 如果新确定了最短路径的顶点 j 加入后, 使 v_1 到 v_i 的距离值比原来的距离值更小, 则修改 v_i 的距离值 $\Pi(v_i) = \min(\Pi(v_i), \Pi(j) + w_{ji})$

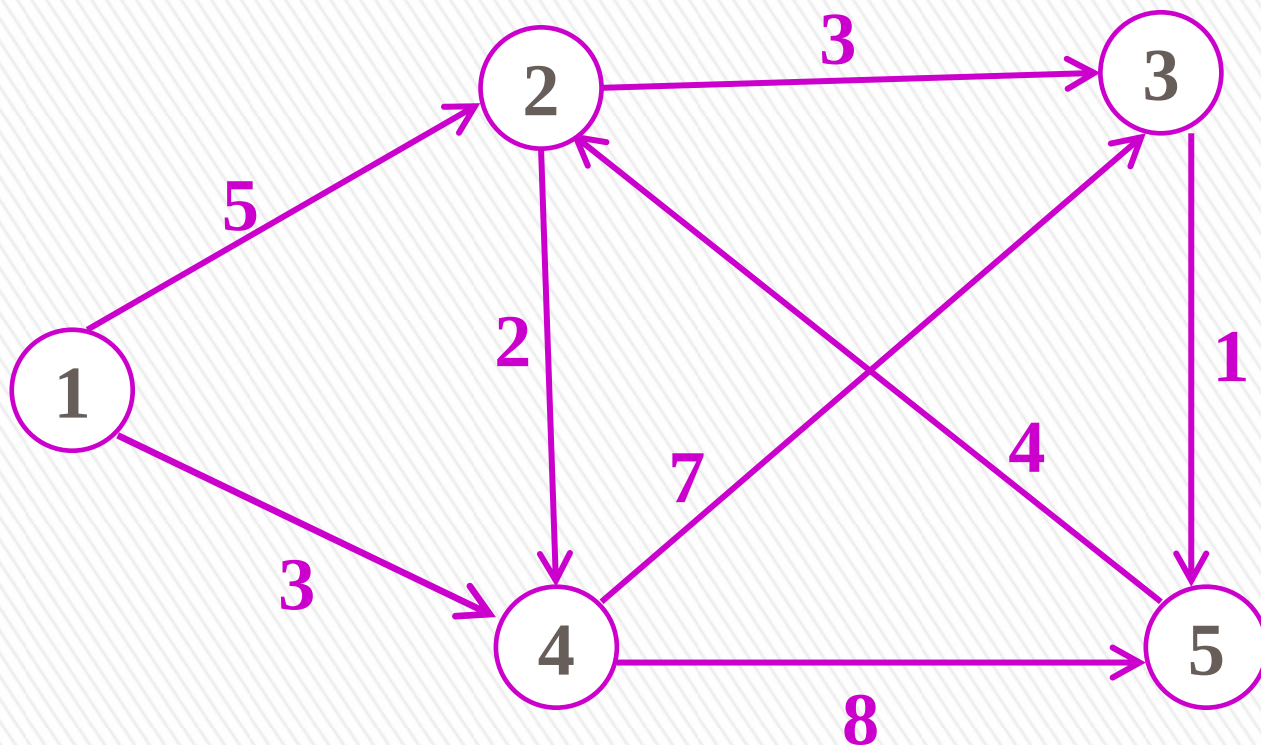
并且将 j 赋给 $Q(i)$:

1	5	1	5	3	2
---	---	---	---	---	---

表明: $Q(6) = v_2$, $Q(2) = v_5$, $Q(5) = v_3$, $Q(3) = v_1$, 所以 v_1 到 v_6 的最短路径为 $(v_1, v_3, v_5, v_2, v_6)$

Dijkstra算法

■ 对有向图完全适用



Dijkstra算法

$P = \{1\}, T = \{2, 3, 4, 5\}$, 且 $S_1 = 0, S_2 = w\langle 1, 2 \rangle = 5$

$S_3 = w\langle 1, 3 \rangle = \infty, S_4 = w\langle 1, 4 \rangle = 3, S_5 = w\langle 1, 5 \rangle = \infty$

$k = 4, P = \{1, 4\}, T = \{2, 3, 5\}$

$S_2 = \min\{S_2, S_4 + w\langle 4, 2 \rangle\} = \min\{5, 3 + \infty\} = 5$

$S_3 = \min\{S_3, S_4 + w\langle 4, 3 \rangle\} = \min\{\infty, 3 + 7\} = 10$

$S_5 = \min\{S_5, S_4 + w\langle 4, 5 \rangle\} = \min\{\infty, 3 + 8\} = 11$

$k = 2, P = \{1, 4, 2\}, T = \{3, 5\}$

$S_3 = \min\{S_3, S_2 + w\langle 2, 3 \rangle\} = \min\{10, 5 + 3\} = 8$

$S_5 = \min\{S_5, S_2 + w\langle 2, 5 \rangle\} = \min\{11, 5 + \infty\} = 11$

$k = 3, P = \{1, 4, 2, 3\}, T = \{5\}$

$S_5 = \min\{S_5, S_3 + w\langle 3, 5 \rangle\} = \min\{11, 8 + 1\} = 9$

显然: $S_1 = 0, S_2 = 5, S_3 = 8, S_4 = 3, S_5 = 9$

Dijkstra算法

- 什么时候“停止计算”？
- 计算某两点 ($a \rightarrow b$) 之间的最短路
 - Dijkstra算法，扩展到 b 时即可停止；
- 计算某点到其它各个节点的最短路
 - Dijkstra算法，扩展完所有节点后停止；
- 两者复杂程度在最坏情况下不存在本质区别

最短路径模型

■ 最短路径可研究的情况包括

	边权为 正数	边权为 1	边权为 任意实数
某点到其他点 (指定的两点)	Dijkstra		
任意两点之间			

关于 Dijkstra 算法，以下哪项是正确的？

- ☐ A Dijkstra 算法使用贪心策略来解决单源最短路径问题，并且能够处理负权重的边。
- ☐ B Dijkstra 算法在无向图中无法运行，只能用于有向图。
- ☒ C Dijkstra 算法如果使用简单的方法实现，时间复杂度为 $O(n^2)$ 。
- ☐ D Dijkstra 算法总是能够找到图中所有顶点之间的最短路径。

提交

最短路径模型

■ 最短路径可研究的情况包括

	边权为 正数	边权为 1	边权为 任意实数
某点到其他点 (指定的两点)	Dijkstra	?	
任意两点之间			

讨论权值为1的情况下某点到其他各个节点最短路径的问题

最短路径

- 当边权为1时 v_1 到各点的最短路

广度优先搜索法

计算复杂度 $O(m)$, m 为边的数目

最短路径模型

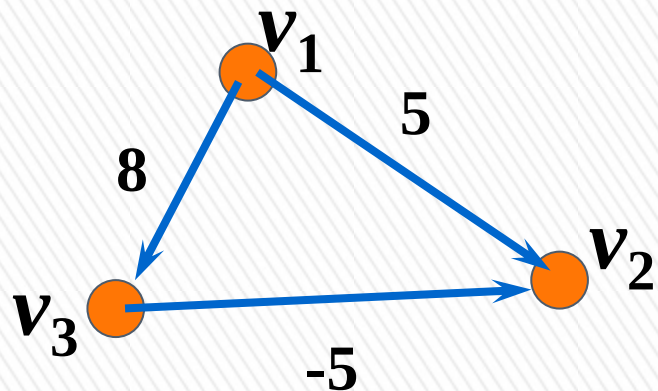
■ 最短路径可研究的情况包括

	边权为 正数	边权为 1	边权为 任意实数
某点到其他点 (指定的两点)	Dijkstra	BFS	?
任意两点之间			

讨论权值为任意实数的情况下某点到其他各个节点的最短路径

最短路径

- 当边权为任意实数时 v_1 到各点的最短路
Dijkstra可直接适用吗？



Dijkstra算法中， $\pi(2) = 5$ ，而正确解是 $\pi(2) = 3$
Ford给出算法解决这一问题

最短路径

■ 当边权为任意实数时 v_1 到各点的最短路

注意：

如果没有“负长回路”，可证明最短路径中不包含正权回路，此时**最短路径不能有回路**，即路径长度最长为 $|V|-1$ 。

Ford算法思路：根据路径长度从0到 $|V|-1$ ，逐步迭代（更新后继节点）得到在当前路径长度范围内 v_1 到各点的最短路，并最终得到 v_1 到各点的最短路。

最短路径

■ 当边权为任意实数时 v_1 到各点的最短路

Ford算法

问题:

(1) 负长回路情况?

无法终止

(2) 无负长回路情况下最多迭代次数?

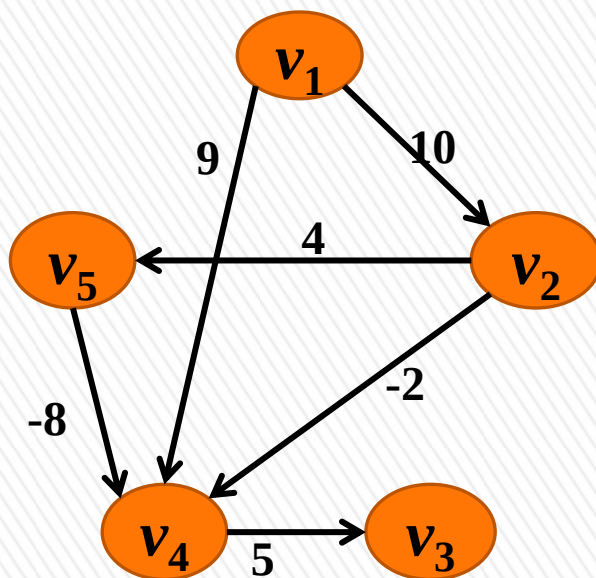
从 v_1 到 v_i 的最短路长度是 $\pi(i)$ 的下界, 经过 $n - 1$ 次迭代后 $\pi(i)$ 将保持不变。

最短路径

■ 当边权为任意实数时 v_1 到各点的最短路

Ford算法实例

起点	终点	权值
1	2	10
4	3	5
1	4	9
2	4	-2
5	4	-8
2	5	4



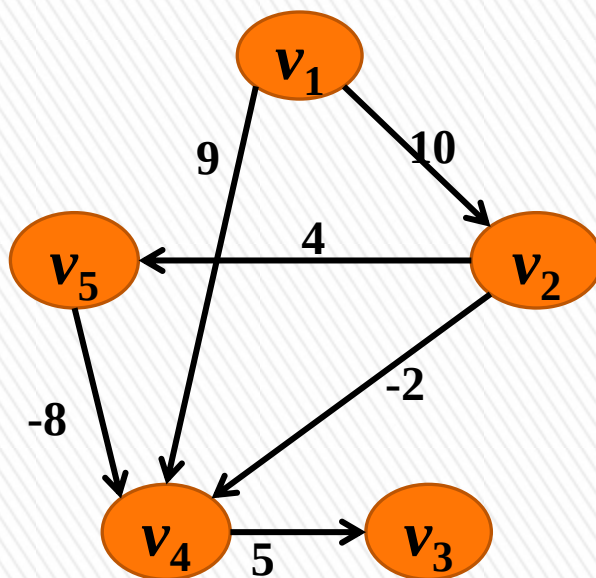
$\pi(1)$	$\pi(2)$	$\pi(3)$	$\pi(4)$	$\pi(5)$	k
0	∞	∞	∞	∞	0
0	10	∞	9	∞	1

最短路径

■ 当边权为任意实数时 v_1 到各点的最短路

Ford算法实例

起点	终点	权值
1	2	10
4	3	5
1	4	9
2	4	-2
5	4	-8
2	5	4



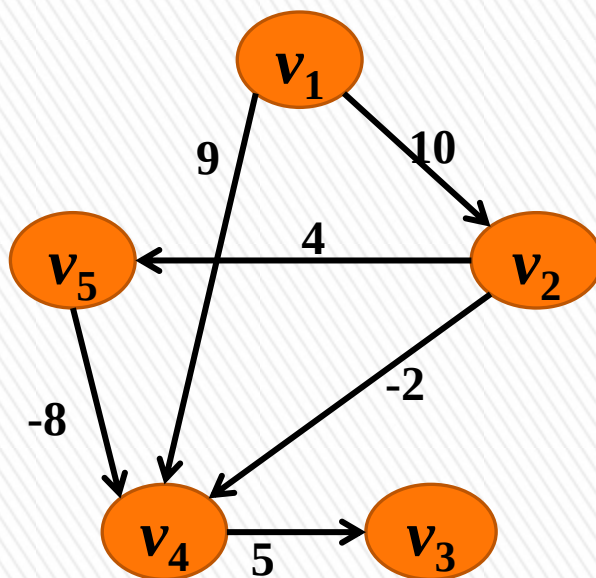
$\pi(1)$	$\pi(2)$	$\pi(3)$	$\pi(4)$	$\pi(5)$	k
0	∞	∞	∞	∞	0
0	10	∞	9	∞	1
0	10	14	8	14	2

最短路径

■ 当边权为任意实数时 v_1 到各点的最短路

Ford算法实例

起点	终点	权值
1	2	10
4	3	5
1	4	9
2	4	-2
5	4	-8
2	5	4



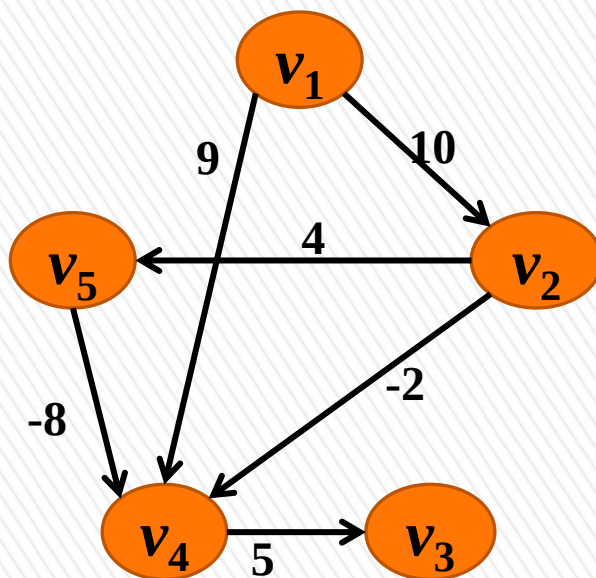
$\pi(1)$	$\pi(2)$	$\pi(3)$	$\pi(4)$	$\pi(5)$	k
0	∞	∞	∞	∞	0
0	10	∞	9	∞	1
0	10	14	8	14	2
0	10	13	6	14	3

最短路径

■ 当边权为任意实数时 v_1 到各点的最短路

Ford算法实例

起点	终点	权值
1	2	10
4	3	5
1	4	9
2	4	-2
5	4	-8
2	5	4

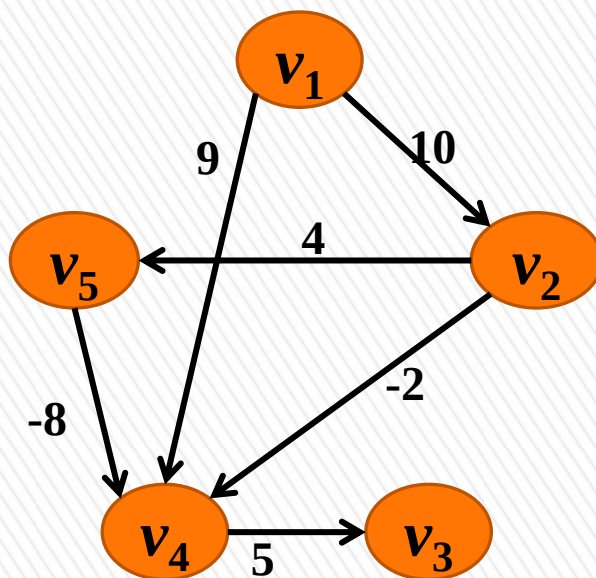


$\pi(1)$	$\pi(2)$	$\pi(3)$	$\pi(4)$	$\pi(5)$	k
0	∞	∞	∞	∞	0
0	10	∞	9	∞	1
0	10	14	8	14	2
0	10	13	6	14	3
0	10	11	6	14	4

最短路径

■ 当边权为任意实数时 v_1 到各点的最短路

Ford算法实例



$\pi(1)$	$\pi(2)$	$\pi(3)$	$\pi(4)$	$\pi(5)$	k
0	∞	∞	∞	∞	0
0	10	∞	9	∞	1
0	10	14	8	14	2
0	10	13	6	14	3
0	10	11	6	14	4
0	10	11	6	14	5

最短路径

■ 当边权为任意实数时 v_1 到各点的最短路

Ford算法

① 赋初值 $\pi(1) = 0, \pi(i) = \infty, i = 2, 3, \dots, n$.

② 更新 $\pi(i), i = 2, 3, \dots, n$.

$$\pi(i) = \min \{ \pi(i), \min \{ \pi(j) + w_{ji} \mid j \in i \text{ 的直接前驱集} \} \}.$$

③ 若所有的 $\pi(i)$ 都无变化, 停止; 否则转向②.

②中每迭代一次, 路径长度范围加1

算法复杂性:

在算法的每一步, $\pi(i)$ 都是从 v_1 到 v_i 的最短路长度的上界.

步骤2需要 m 次加法和比较

那么迭代多少次可以停止呢?

最短路径

■ 当边权为任意实数时 v_1 到各点的最短路

Ford算法

问题:

(1)负长回路情况?

无法终止

(2)无负长回路情况下最多迭代次数?

从 v_1 到 v_i 的最短路长度是 $\pi(i)$ 的下界, 经过 $n - 1$ 次迭代后 $\pi(i)$ 将保持不变。

最坏情况下, Ford算法的计算复杂度为 $O(mn)$

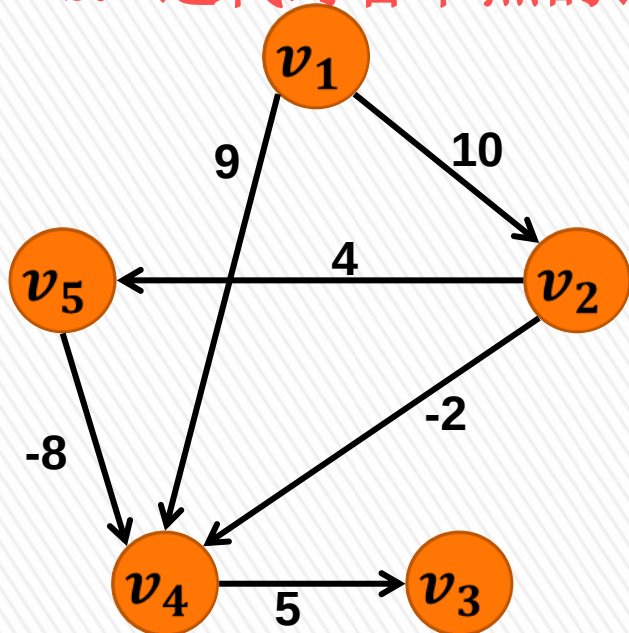
(3)如何使算法尽快终止?

最短路径

问题： 如何使算法尽快收敛？

a. 初始值很重要

b. 迭代时各节点的计算顺序很重要



使用Dijkstra算法得到的结果（忽略负边）

$\pi(2) = 10, \pi(3) = 14, \pi(4) = 9, \pi(5) = 14$

节点序为 v_4, v_2, v_3, v_5

$\pi(1)$	$\pi(4)$	$\pi(2)$	$\pi(3)$	$\pi(5)$	k
0	9	10	14	14	0
0	6	10	11	14	1

最短路径模型

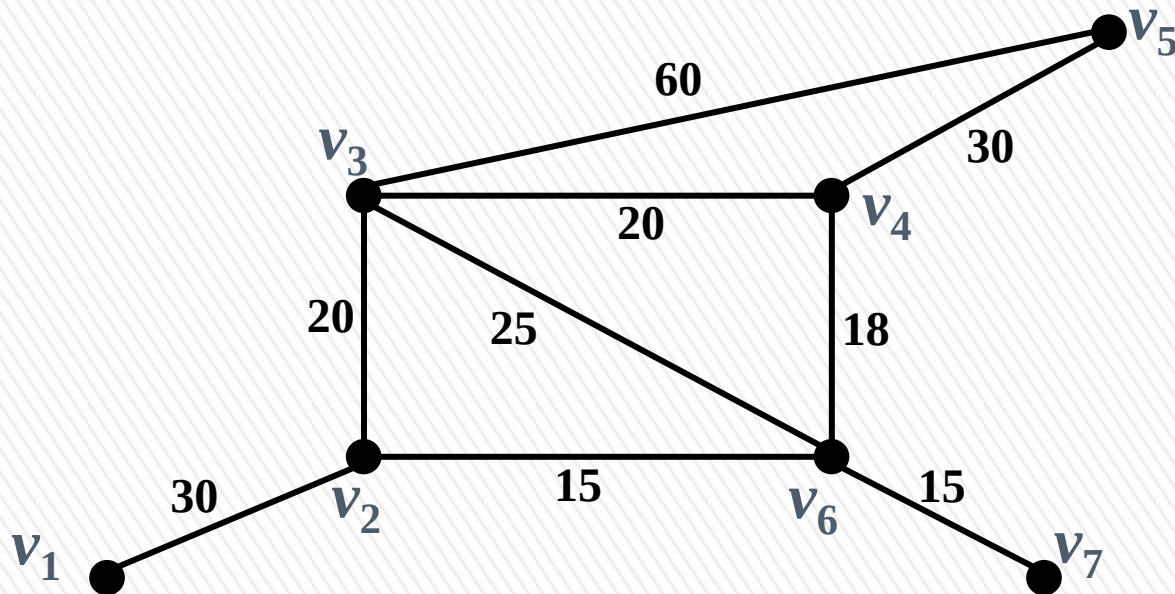
■ 最短路径可研究的情况包括

	边权为 正数	边权为 1	边权为 任意实数
某点到其他点 (指定的两点)	Dijkstra	BFS	Ford (负长回路不终止)
任意两点之间			

选址问题

■ 问题

现准备在 n 个居民点 $v_1, v_2, \dots, v_n$ 中设置一银行。问设在哪个点，可使最大服务距离最小？若设置两个银行，问设在哪两个点？



选址问题

■ 模型建立与求解

求出任意两个居民点 v_i, v_j 之间的最短距离，并用 d_{ij} 表示。

(1) 设置一个银行，银行设在 v_i 点的最大服务距离为

$$d_i = \max_{1 \leq j \leq n} \{d_{ij}\}, i = 1, 2, \dots, n.$$

求 k , 使 $d_k = \min_{1 \leq i \leq n} \{d_i\}$.

即若设置一个银行，则银行设在 v_k 点，可使最大服务距离最小。

(2) 设置两个银行，假设银行设在 v_s, v_t 点使最大服务距离最小。

记 $d(i, j) = \max_{1 \leq k \leq n} \{\min\{d_{ik}, d_{jk}\}\}$.

则 s, t 满足: $d(s, t) = \min_{1 \leq i < j \leq n} \{d(i, j)\}$.

最短路径模型

■ 最短路径可研究的情况包括

	边权为 正数	边权为 1	边权为 任意实数
某点到其他点 (指定的两点)	Dijkstra	BFS	Ford (负长回路不终止)
任意两点之间	?	?	?

讨论任意两点之间的最短路径

任意两点之间的最短路

解法一：

- 实际是要求出图的中心，可以化为一系列求最短路问题。
- 先求出 v_1 到其他各顶点的最短路，令 $D(v_1) = \max \{d_1, d_2, \dots, d_7\}$ ，表示若银行建在 v_1 ，则离银行最远的小区距离为 $D(v_1)$ ；
- 再依次计算 $v_2, v_3, \dots, v_7$ 到其余各点的最短路，
- 类似求出 $D(v_2), D(v_3), \dots, D(v_7)$ ，
 $D(v_i)(i = 1, 2, \dots, 7)$ 中最小者即为所求。

选址问题

	v_1	v_2	v_3	v_4	v_5	v_6	v_7	$D(v_i)$
v_1	0	30	50	63	93	45	60	93
v_2	30	0	20	33	63	15	30	63
v_3	50	20	0	20	50	25	40	50
v_4	63	33	20	0	30	18	33	63
v_5	93	63	50	30	0	48	63	93
v_6	45	15	25	18	48	0	15	48
v_7	60	30	40	33	63	15	0	63

由于 $D(v_6) = 48$ 最小，所以银行建在 v_6 ，此时离银行最远的小区 v_5 距离为 48。

任意两点之间的最短路

■ Floyd算法 1962提出

逐个节点试探法。

- 初始时设置一个 n 阶方阵，令其对角线元素为0，若存在边 $\langle V_i, V_j \rangle$ ，则对应元素为权值；否则为 ∞ ；
- 逐步试着在原直接路径中增加中间节点，若加入中间点后路径变短，则修改之；否则，维持原值；
- 所有节点试探完毕，算法结束。

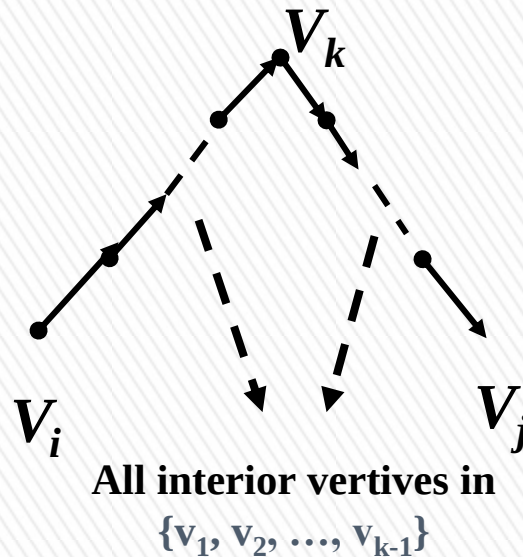
路径的矩阵求解

$$P_{ij}^{(k)} = p_{ij}^{(k-1)} \vee (p_{ik}^{(k-1)} \wedge p_{kj}^{(k-1)})$$

Case 1



Case 2



路径的矩阵求解

Warshall's Algorithm

```
 $P = [a_{ij}]_{n \times n}$   
for ( $k = 1; k \leq n; k++$ )  
{  
    for ( $i = 1; i \leq n; i++$ )  
    {  
        for ( $j = 1; j \leq n; j++$ )  
             $P_{ij} = P_{ij} \vee (P_{ik} \wedge P_{kj});$   
    }  
}
```

算法复杂度: $2n^3 = O(n^3)$

任意两点之间的最短路

■ Floyd算法 1962提出

设 $A = (a_{ij})_{n \times n}$ 为赋权图 $G = (V, E, F)$ 的权矩阵, d_{ij} 表示从 v_i 到 v_j 点的距离。

① 赋初值。

对所有 i, j , $d_{ij} = a_{ij}$, $k = 1$ 。转向②。

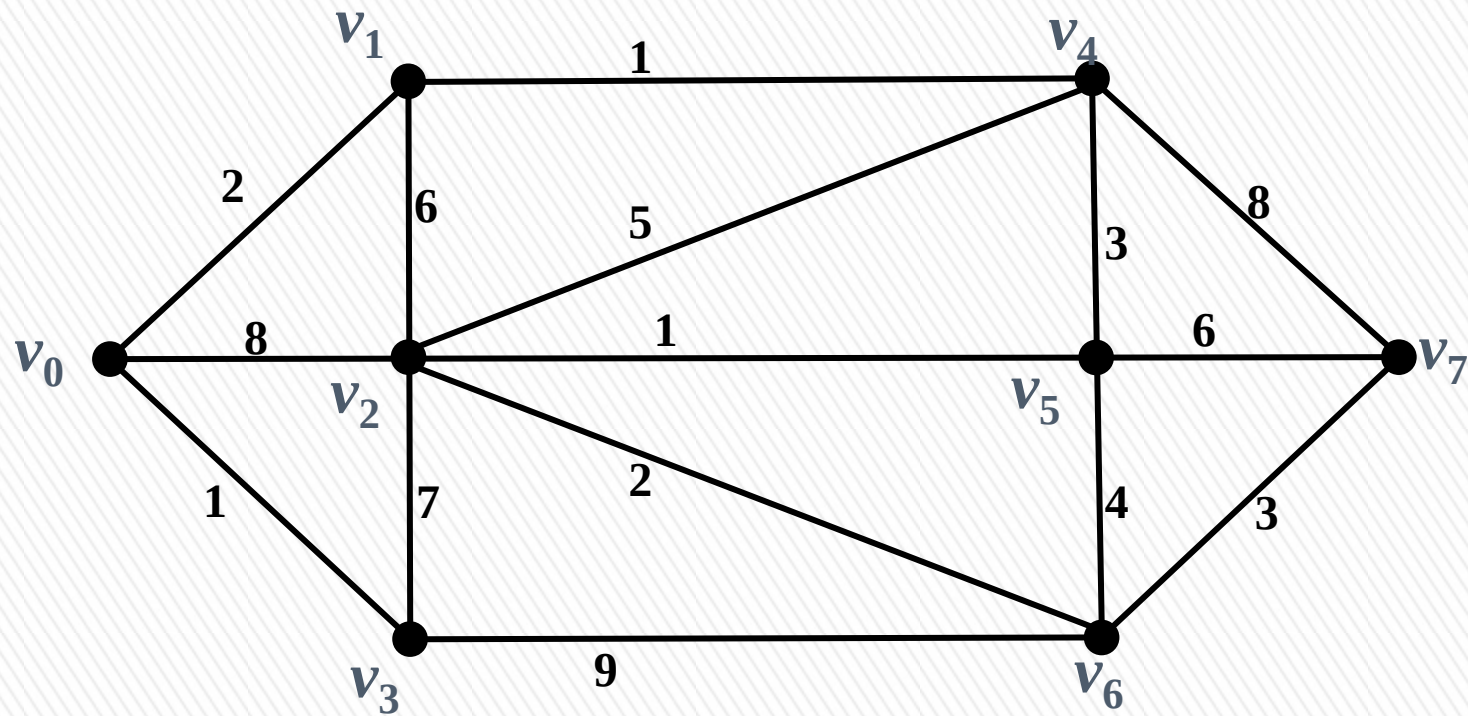
② 更新 d_{ij} 。

对所有 i, j , 若 $d_{ik} + d_{kj} < d_{ij}$, 则令 $d_{ij} = d_{ik} + d_{kj}$, 转向③;

③ 终止判断。

若 $k = n$ 终止; 否则令 $k = k + 1$, 转向②。

Floyd算法



关于 Floyd 算法的描述，下面哪项是正确的？

- ☐ A Floyd 算法主要用于寻找图中两个顶点之间的最长路径。
- ☒ B Floyd 算法可以用来处理带有负权重的边，但不能处理带有负权重环的图。
- ☒ C Floyd 算法的时间复杂度是 $O(n^3)$ 。
- ☐ D Floyd 算法只适用于无向图。

提交

最短路径模型

■ 最短路径可研究的情况包括

	边权为 正数	边权为 1	边权为 任意实数
某点到其他点 (指定的两点)	Dijkstra	BFS	Ford (负长回路不终止)
任意两点之间	Floyd	Floyd	Floyd (负长回路不正确)

本堂课小结

■ 旅行商问题

- 分支定界法
- 便宜算法

■ 分支定界法、贪心算法

■ 最短路径

- **Dijkstra**（正权）、**BFS**（权为1）
- **Ford**（无负回路）
- **Floyd**算法（任意两点）